



浙江财经大学

本科生毕业论文(设计)

题目：基于深度学习的管道缺陷检测系统的设计与开发

学生姓名：	卫亦扬
学 号：	210110900425
指导教师：	朱凌
所在学院：	信息技术与人工智能学院
专业名称：	软件工程
班 级：	21 软件 2 班

2025 年 4 月

声明及论文使用的授权

本人郑重声明所呈交的论文是我个人在导师的指导下独立完成的。除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写的研究成果。

论文作者签名：

年 月 日

本人同意浙江财经大学有关保留使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以上网公布全部内容，可以采用影印、缩印或其他复制手段保存论文。

论文作者签名：

年 月 日

基于深度学习的管道缺陷检测系统的设计与开发

摘要：管道的健康状况对城市基础设施的稳定运行至关重要，及时、准确地检测管道缺陷能够有效降低维护成本，提高管道运行的安全性和可靠性。针对管道缺陷检测任务，传统人工方法存在检测效率低、成本高昂及主观判断偏差等局限性，而基于深度学习的智能检测技术通过算法优化有效提升了检测精度与效率。本研究系统探讨了深度学习在管道缺陷检测中的应用机理，提出一种改进型 YOLOv11 算法，通过优化网络结构及特征融合机制，有效提升了复杂管道缺陷的识别精度与检测速度。实验结果表明，该算法在多个关键评价指标（包括 mAP、召回率和精确率）上均优于现有方法，能够高效识别裂缝、变形、堵塞等多种管道缺陷。基于改进的深度学习算法，本研究进一步开发了一套智能管道缺陷检测系统，实现了实时监控、目标检测、可视化展示、检测报告生成等功能。系统采用网页前后端架构，支持在线检测与结果反馈，满足实际工程应用需求。该研究不仅为城市管网维护提供了一种智能化解决方案，也为深度学习在工业检测领域的应用提供了重要参考。

关键词：管道缺陷检测；目标检测；深度学习；YOLOv11；智能检测系统

The Design and Development of a Pipeline Defect Detection System Based on Deep Learning

Abstract: The health condition of pipelines is crucial for the stable operation of urban infrastructure, and timely, accurate defect detection can effectively reduce maintenance costs while improving safety and reliability. Traditional manual inspection methods face limitations such as low efficiency, high costs, and subjective judgment biases, whereas deep learning-based intelligent detection techniques enhance accuracy and efficiency through algorithmic optimization. This study systematically explores the application mechanism of deep learning in pipeline defect detection and proposes an improved YOLOv11 algorithm, which boosts the recognition accuracy and detection speed for complex pipeline defects by

optimizing network architecture and feature fusion mechanisms. Experimental results demonstrate that the algorithm outperforms existing methods across key metrics (including mAP, recall, and precision) in identifying various defects such as cracks, deformations, and blockages. Building on this enhanced algorithm, the research further develops an intelligent pipeline defect detection system with real-time monitoring, object detection, visualization, and automated report generation. The system adopts a web-based front-end and back-end architecture, supporting online detection and feedback to meet practical engineering needs. This study not only provides a smart solution for urban pipeline maintenance but also offers valuable insights for applying deep learning in industrial inspection.

Key words: Pipeline defect detection; Object detection; Deep learning; YOLOv11; Intelligent detection system

目 录

摘 要	I
Abstract	I
1 引 言	1
1.1 课题背景及意义	1
1.2 论文组织架构	2
2 相关工作	3
2.1 管道缺陷检测技术发展概述	3
2.2 YOLO 系列模型在缺陷检测中的应用与演进	3
2.3 其他相关算法与技术	5
2.3.1 数据管理与智能决策支持系统	5
2.3.2 新型传感器与管道检测机器人技术	5
3 数据处理和任务概要	6
3.1 数据来源与预处理	6
3.1.1 数据概况	6
3.1.2 数据集的构建与预处理	6
3.2 任务概要	7
3.2.1 算法任务概要	7
3.2.2 系统设计概要	7
4 基于深度学习的管道缺陷检测算法	9
4.1 YOLOv11 网络结构	9
4.1.1 模型介绍和网络结构	9
4.1.2 改进的 YOLOv11 网络结构	10
4.2 AIFI 模块设计与应用	12
4.2.1 模块优势	12
4.2.2 模块构建	13
4.3 C3K2_context_guided 模块设计与应用	14
4.3.1 模块优势	14
4.3.2 模块构建	15
4.4 CBAM 模块设计与应用	16
5 实验与模型性能评估	18
5.1 训练过程与参数设置	18

5.2	性能评价指标与测试结果	19
5.3	消融实验结果	20
5.4	对比实验与分析	21
6	系统设计与实现	23
6.1	系统分析	23
6.1.1	应用背景分析	23
6.1.2	市场需求分析	23
6.1.3	可行性分析	24
6.1.4	功能需求分析	26
6.2	系统设计	27
6.2.1	技术路线设计	27
6.2.2	功能模块设计与实现	27
6.2.3	数据字典设计	37
6.2.4	数据库设计	38
6.3	系统实现与展示	39
6.3.1	开发环境	39
6.3.2	系统主页面	39
6.3.3	实时监控模块	39
6.3.4	报警信息管理模块	40
6.3.5	设备管理模块	41
6.3.6	场站管理模块	42
结 论		43
参考文献		44
致 谢		48

1 引言

1.1 课题背景及意义

管道系统是保障城市正常运行的关键通道。然而，随着管道运行年限的增长和外部环境的变化，管道系统日益面临各种潜在风险和缺陷问题，如裂缝、渗漏、变形、堵塞以及沉积物堆积等^[1-4]。如果这些问题不能被及时发现并加以修复，不仅可能导致城市供水中断、道路塌陷等严重后果，还会引发环境污染与资源浪费，甚至危及公共安全。

传统的管道检测方法，如人工检查和闭路电视（CCTV）检测，依赖于人工操作，不仅耗时耗力，而且容易受到操作员主观判断的影响，存在效率低、误差大的问题^[5]。如何利用现代自动化、智能化手段提升检测的效率和准确性，成为城市管道管理维护中的重要研究方向。

近年来，管道缺陷检测的难题涌现出多种创新性的解决方案。深度学习作为一种强大的机器学习方法，能够自动提取图像中的复杂特征，显著提高检测精度和效率^[6]，在交通、安防、医疗等多个领域已展现出广泛应用前景。在管道无损检测技术研究中，以深度学习框架为理论基础的智能识别算法逐渐成为主流解决方案，并取得了显著成效。

其中，YOLO 系列算法以其高效的检测速度和较强的定位能力，受到研究者和工程实践者的广泛关注。近年来对 YOLO 架构的持续优化推动了其在工业检测领域的广泛应用，如改进的 YOLOv8 算法在管道缺陷检测中展现了高效性和实时性，能够有效应对复杂环境下的目标检测问题^[7]。此外，基于深度学习的检测系统还能够显著降低人工干预的需求，减少人力成本和潜在的安全风险^[7-8]。基于此，研究者开始尝试进一步提升检测精度与适应性，以满足复杂场景下的实际需求。

尽管如此，当前的研究仍存在一定局限性。例如，在弱光、模糊、水雾等复杂环境下，模型的检测准确率仍有待提升；部分模型缺乏良好的泛化能力，难以适应多源异构数据；此外，现有系统多局限于研究阶段，尚未形成稳定、易用的工程化产品。

综上所述，基于深度学习的管道缺陷检测不仅有助于提高城市基础设施的管理水平和智能化程度，也为推动人工智能技术在城市运维中的落地应用提供了新思路。本研究聚焦于深度学习目标检测领域的研究进展，针对现有 YOLOv11 模型的性能优化提出改进方案，并构建一套基于深度学习的管道缺陷检测系统，旨在实现检测精度与工程应用价值的同步提升。

1.2 论文组织架构

本文围绕基于深度学习的管道缺陷检测展开研究，设计并实现了一种基于改进 YOLOv11 的高效目标检测算法，并构建了完整的智能检测系统，实现了从图像采集、缺陷识别到可视化展示与报告生成的全流程智能化。在算法设计中，本文引入了自适应特征交互模块 AIFI^[9]，上下文引导模块 C3K2_context_guided^[10]以及注意力机制 CBAM^[11]，有效提升了对多尺度目标和复杂环境的感知能力，从而提高检测准确率。

本文的其余章节组织结构如下：

第二节将对管道缺陷检测领域相关的传统方法与深度学习方法进行梳理和分析；第三节将阐述实验数据集的获取方式及研究目标、系统设计要点；第四节将详细阐述本文所提出的改进 YOLOv11 算法的整体结构与工作机制，分析模型的创新点和技术优势；第五节将展示并分析算法训练结果和评价指标；第六节将介绍基于算法的智能管道缺陷检测系统的构建过程；最后通过系统梳理核心成果，总结关键发现，并基于现有技术瓶颈分析，探讨了后续改进策略及潜在研究方向。

2 相关工作

2.1 管道缺陷检测技术发展概述

管道检测技术的发展始于 20 世纪中叶，最初主要依赖于人工检查和简单的物理方法。早期的检测手段包括目视检查、敲击测试等^[7]。这些方法虽然操作简便，但效率低下，且容易受到操作者经验和主观判断的影响，存在较大误差和遗漏率。随着城市基础设施的不断扩张与复杂化，这类传统检测手段逐渐无法满足现代化检测的需求。

自 20 世纪后期，无损检测技术（NDT）逐步成为管道完整性评估领域的关键手段。典型的 NDT 方法包括超声波检测、射线检测、漏磁检测等，可在不破坏管道结构的前提下检测内部缺陷^[12]。例如，漏磁检测技术通过检测管道壁的磁场变化来识别腐蚀和裂纹，广泛应用于油气管道的检测^[13]。

进入 21 世纪，随着计算机技术、传感器技术及机器人技术的迅速发展，智能检测逐步取代传统方式。例如，基于机器视觉的检测系统通过分析 CCTV 视频或图像数据，能够自动识别管道缺陷^[14]。此外，探地雷达技术被用于检测浅层地下管线的暗管和暗沟问题^[15]，尽管其在某些材料的管道检测中存在局限性。

近年来，深度学习技术在管道缺陷检测中得到了广泛应用。同时，多技术融合成为发展趋势，例如将激光扫描技术、3D 建模与 CCTV 检测相结合，以提高检测的精度和全面性^[16]。

2.2 YOLO 系列模型在缺陷检测中的应用与演进

作为目标检测领域的代表性模型，YOLO 算法系列自 YOLOv3 至 YOLOv11 经历了多次迭代优化，其版本更新显著提升了检测精度与计算效率。例如，通过采用多尺度特征融合策略，YOLOv3 算法在小尺寸目标的识别精度方面取得显著改进；YOLOv4 和 YOLOv5 则进一步优化了网络结构和损失函数，提高了检测精度和模型的计算效率^[17]。YOLOv7 在继承 YOLOv5 的基础上，通过优化网络结构、引入新的特征提取模块（如注意力机制）和损失函数，大幅提升了检测的精度和速度。特别是在多尺度目标检测和小目标检测方面，YOLOv7 展现了显著的优势^[18]。YOLOv11 模型在多维度实现了性能突破。具体而言，YOLOv11m 在交通标志数据集上达到了 0.795 的 mAP50-95 分数，同时保持了 2.4ms 的平均推理时间和 38.8Mb 的模型大小^[19]。这表明 YOLOv11 在实时性和

准确性之间取得了良好的平衡，适用于多种复杂场景。YOLOv11 通过引入新的特征提取模块和优化策略，能够更高效地处理复杂场景中的目标检测任务^[19-20]。在管道缺陷检测中，YOLOv11 可以应用于实时检测和定位缺陷，表现出优异的性能。

作为 YOLO 系列的典型代表算法，YOLOv5 与 YOLOv7 凭借其目标识别性能优势，被广泛应用于管道缺陷检测领域。为了使准确率更高，研究者们提出了多种解决方案。例如，Jia 等人^[18]提出了一种改进的 YOLOv5 模型，通过引入 CSSPPF 模块和 CBAM 注意力机制，增强了多尺度特征学习能力，提升了对小目标的检测精度。Wang 等人^[21]则开发了一种改进的 YOLOv5s 算法，通过引入卷积注意力机制和知识蒸馏技术，进一步提高了复杂背景下的检测准确性，并减少了模型的计算量。此外，Yan 等人^[22]提出了一种基于改进 YOLOv7 的异常信号检测方法（YOLO-PD），通过引入精细通道注意力模块和多尺度特征融合技术，显著提升了多尺度目标检测的精度。Denglian Yang 等人^[23]提出了一种基于改进 YOLOv8 架构的自动金属管道缺陷检测方法——OFG-YOLO，该方法通过调整卷积结构、整合空间金字塔池化模块与聚焦调制技术等优化了模型的特征提取和识别能力。此外，研究还结合了 Gold-YOLO 的聚集与分布机制，进一步提升了模型对多尺度特征的融合效果。这些改进表明，通过优化 YOLO 模型的结构，可以有效提升管道缺陷检测的性能。

为了进一步提升 YOLO 算法在管道缺陷检测中的表现，研究者们尝试将注意力机制和特征融合技术引入到 YOLO 模型中。Peng 等人^[24]提出了一种基于 CBAM 注意力机制的 YOLOv5 改进算法，用于电厂油库管道泄漏检测，通过加强模型对关键特征的关注，提升了在复杂背景下的检测效果。Tang 等人^[25]在 PCB 表面缺陷检测中，结合 Swin Transformer 和注意力机制，设计了一种改进的 YOLOv5 算法（PCB-YOLO），不仅提高了小目标的检测精度，还在模型体积和计算效率上做出了优化，使其更加适合实际工业场景的部署。Wen 等人^[26]通过引入结构重参数化技术和 Ghost 卷积，开发了一种轻量化的管道缺陷检测方法，在保持高精度的同时显著提升了检测速度。Wang 等人^[21]提出了一种基于 YOLOv5s 的改进算法，通过引入卷积注意力机制和知识蒸馏技术，提升了复杂背景下的检测精度。Zhu 等人^[27]则在 YOLOv7 中引入了特征复用和结构再参数化模块，提高了检测速度和精度。

2.3 其他相关算法与技术

2.3.1 数据管理与智能决策支持系统

随着智能化检测技术的广泛应用，检测系统不再局限于图像识别本身，而是朝着数据管理和智能决策支持方向延伸。当前管道检测技术已逐步发展为集异常定位与数据智能管理于一体的综合系统。这种系统通过实时数据传输和分析，能够快速响应潜在的管道问题，减少维修成本和时间^[19]。

此外，智能决策系统通过机器学习算法对检测数据进行分析，系统能够预测管道的潜在风险，并为维护工作提供优化建议^[28]。这种从“检测”到“预测”的转变，是深度学习与工业智能融合的核心体现。

2.3.2 新型传感器与管道检测机器人技术

现代管道检测技术还依赖于先进传感器和检测机器人系统的支撑。各类高灵敏度、抗干扰能力强的传感器被广泛应用于物理状态检测、气体泄漏监测等任务中^[29]。例如，电化学传感器、光纤传感器可用于检测特定化学物质，提升安全预警能力，减少事故风险。

与此同时，智能检测机器人也在管道检测中得到应用与推广。这些机器人可自主在狭窄、弯曲、潮湿的管道中移动，搭载摄像头、雷达和传感器进行多模态信息采集，实现全自动化缺陷检测、路径规划和结果上传^[30-31]。尤其在无法人工进入的管道中，机器人系统的优势尤为明显。

3 数据处理和任务概要

3.1 数据来源与预处理

3.1.1 数据概况

实验数据选自两个公共数据库，具体包括 Sewer-ML Dataset 和 Pipe Dataset（V2）。

(1)Sewer-ML Dataset^[32]

该数据集由丹麦奥尔堡大学提供，是目前较为权威的城市排水管道缺陷图像数据集，涵盖裂缝、腐蚀、变形、沉积等多类典型缺陷，所有图像均由 CCTV 机器人采集，并由专业人员进行多层次标注，广泛应用于学术研究和算法训练。

(2)Pipe Dataset（V2）^[33]

该数据集来源于 Roboflow Universe 平台，是一套面向工业金属管道缺陷检测的开源图像集。数据由专业管道维护人员采集并标注，涵盖排水管道、油气管道、蒸馏装置管道等多类应用场景，具有多样化、真实性和复杂性等特点。

从上述两个数据集中选取了部分质量较高的数据，采用 YOLO 格式进行缺陷标注，确保与目标检测算法的高度兼容性。本研究构建的数据集按功能划分为训练集、验证集和测试集三个独立模块。除了用于训练的 8070 张图像外，还有 101 张图像用于测试，100 张用于验证。训练集包括 9821 个腐蚀标签、9543 个堵塞标签、476 个变形标签和 14876 个裂缝标签。其类别分布如表3.1所示。

表 3.1 数据集标签

	训练集	验证集	测试集	全部
腐蚀	9821	63	96	9980
堵塞	9543	85	28	9656
变形	476	42	5	523
裂缝	14876	97	15	14988
全部	34716	287	144	35174

3.1.2 数据集的构建与预处理

为了提升模型的泛化能力与鲁棒性，本研究对原始图像数据进行了统一预处理。首先，将所有图像缩放至 640×640 分辨率。随后，应用多种数据增强策略，包括 50% 概率的水平翻转、50% 概率的垂直翻转、0%-20% 范围内的 Mosaic 增强，从而增强模型对

微弱缺陷的识别能力。通过上述预处理与增强方法，构建了一个样本丰富、结构多样、难度较高的训练数据集。

3.2 任务概要

3.2.1 算法任务概要

在本研究的管道缺陷检测任务中，为实现高精度、强鲁棒性且具有实时响应能力的检测系统，本文设计并优化了基于 YOLOv11 的深度学习模型，并在其基础架构上引入了多个先进模块，旨在提升模型对多类缺陷（堵塞、裂缝、腐蚀、变形）的识别能力，特别是在弱光、模糊、背景干扰等复杂环境下的表现。

首先，模型主体基于 YOLOv11 架构进行构建，该架构本身具备良好的实时性和较强的检测性能，是目标检测领域的代表性单阶段模型。在此基础上，为进一步增强模型的特征表达与上下文感知能力，本文引入了三大关键模块：CBAM 注意力机制模块、AIFI 交互特征增强模块与 C3K2_context_guided 上下文感知模块。

CBAM 模块通过融合通道与空间注意力机制的双分支结构，显著增强了模型对关键区域和显著特征的聚焦能力，同时有效抑制冗余特征的干扰，从而在复杂背景下显著提升检测准确率。AIFI 模块则通过多尺度融合与跨层交互操作，增强了模型对不同尺度缺陷目标（如裂纹与大片腐蚀）的适应性，使得特征在浅层与深层之间实现信息互补，显著提升模型对细粒度目标的辨识能力。C3K2_context_guided 模块则侧重于建模全局上下文信息，其通过引导特征图在提取过程中融合场景语义，有效解决了传统卷积网络在小目标识别中易丢失上下文的问题，从而提升了模型对“腐蚀变形”等形态复杂、背景依赖性强的缺陷类别的识别能力。

综上所述，本研究在 YOLOv11 原有的基础上，系统性地引入多种结构增强与训练优化手段，构建出一个轻量化、高精度、强泛化的管道缺陷检测模型。该模型具备较强的复杂环境适应能力，能够在含有多种缺陷类型的管道图像中精准识别缺陷，为后续的系统集成与实用化部署打下了坚实基础。通过对训练集、验证集和测试集的性能评估，模型在多个评价指标上均表现良好，达成了检测精度与运行效率的双重平衡，实现了自动化、智能化、实时性的管道缺陷识别目标。

3.2.2 系统设计概要

为实现完整的智能检测功能，系统设计包含实时采集、缺陷检测、结果展示与报告生成等多个模块。系统前端通过高分辨率内窥相机实时采集管道内部图像，后端基于改

进的深度学习模型，对输入图像进行快速推理并输出缺陷类型及定位结果。检测结果将通过可视化界面实时展示，突出标注缺陷位置，同时支持异常情况的报警提示功能。系统还集成了缺陷报告生成模块，可根据检测结果生成图文结合的检测报告并支持 PDF 格式导出，便于后续维护决策。前后端通过 Web 系统实现人机交互，支持设备管理、场站管理等，具有良好的用户体验和数据管理能力。整个系统在保证检测精度的同时，兼顾资源消耗与部署便捷性，具备良好的可扩展性，适用于管道巡检任务。

4 基于深度学习的管道缺陷检测算法

4.1 YOLOv11 网络结构

4.1.1 模型介绍和网络结构

YOLOv11 作为实时目标检测领域的最新迭代模型，在检测精度、运算速度与资源效率等方面均实现显著突破，成为当前该技术领域的代表性解决方案。在之前的基础上，YOLOv11 在架构和训练方法上进行了重大改进，使其成为广泛的计算机视觉任务的多功能选择^[34]。

YOLOv11 延续了 YOLO 系列端到端目标检测的架构风格，同时在主干网络、特征融合结构以及检测头等关键模块中融入了 C3K2、C2PSA、PSABlock、SPPF 等优化模块。这些改进显著增强了模型对复杂图像场景的理解和检测精度。

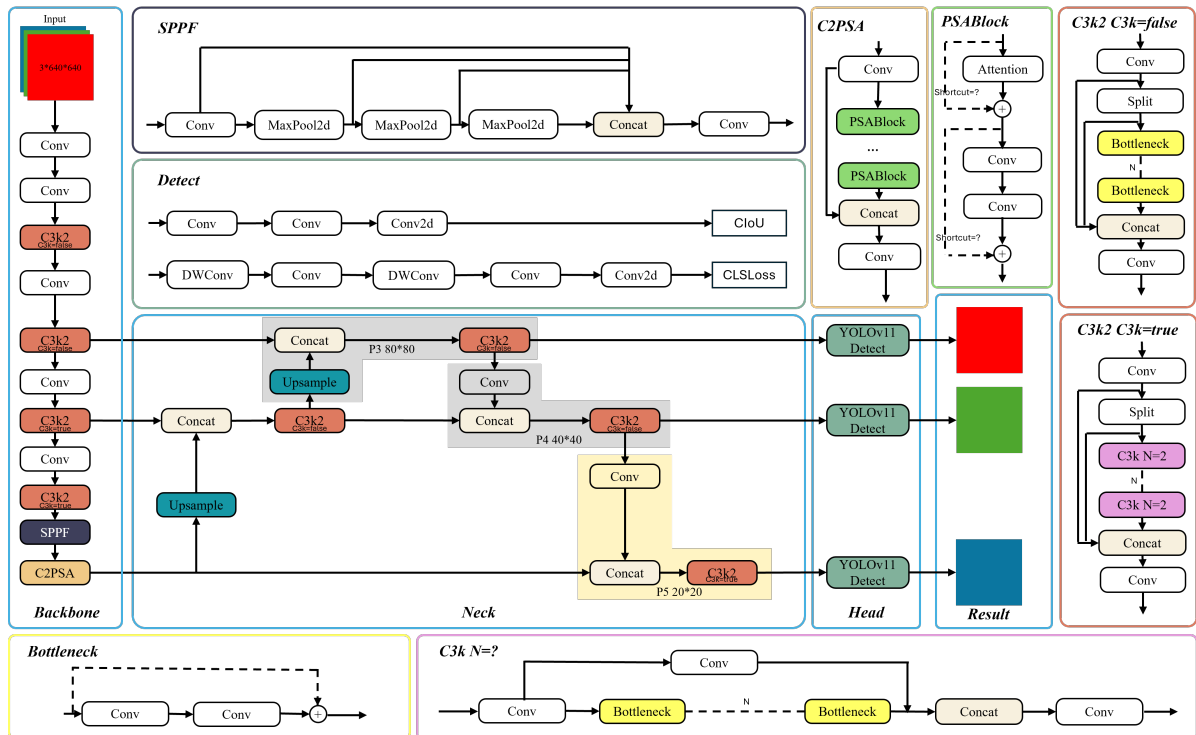


图 4.1 YOLOv11 网络结构图

如图4.1所示，YOLOv11 主要由以下部分组成：

- 输入模块：为下游网络提供经过标准化预处理的规范化的图像数据。

- **Backbone 主干网络**：结构由多层卷积模块与三个 C3K2 模块堆叠构成，逐步提取图像的多尺度语义信息。C3K2 模块通过优化瓶颈结构（Bottleneck），在减少计算量的同时，提升了特征的表达能力。同时在 Backbone 尾部引入了 SPPF（Spatial Pyramid Pooling-Fast）模块，通过三次 MaxPool2d 池化操作增强了感受野，进一步提升模型对全局信息的理解。
- **Neck 特征融合网络**：基于 FPN+PAN 结构，结合上采样与多级特征图 Concat 操作，将来自不同深度层的特征图有效融合。在融合过程的每个阶段均嵌入了 C3K2 模块，从而为检测头提供了更加精细的目标表征。
- **Head 检测头**：负责输出三种不同尺度的检测结果，具体为 80×80 、 40×40 、 20×20 ，分别用于识别大、中、小尺寸目标。检测头由卷积层、深度可分离卷积（DWConv）以及损失函数模块（CIoU Loss、Cls Loss）组成，分别用于完成边界框回归和分类任务。
- **C2PSA 模块与 PSABlock 注意力机制**：YOLOv11 在特征融合末端嵌入 C2PSA 结构，内部集成多个 PSABlock（位置敏感注意力块），通过位置注意机制增强模型对空间局部细节的关注力，有效提升缺陷区域的检测精度。

图像输入尺寸为 $3 \times 640 \times 640$ ，经过多层特征提取后，得到 3 组不同尺度的特征图（P3、P4、P5），分别进入三路 YOLOv11 Detect 检测头，输出对应的缺陷预测框。

4.1.2 改进的 YOLOv11 网络结构

为了进一步提升 YOLOv11 在复杂背景及弱光条件下的检测精度与鲁棒性，本文对其网络结构进行了针对性的改进，具体包括：

加入 CBAM 模块：整合通道与空间注意力机制，有效突出图像中关键区域，提升模型对缺陷区域的关注程度；

加入 AIFI（Adjacent Interactive Feature Integration）模块：在特征融合阶段增强不同层级特征图之间的信息交互，提高小目标识别与上下文理解能力；

加入 C3K2_context_guided 模块：通过替换原有 C3K2 结构，提升模型对局部上下文信息的建模能力，进一步增强缺陷区域的判别性。

通过上述结构增强，改进后的 YOLOv11 不仅提升了检测精度，尤其对微小缺陷具备更强鲁棒性，同时保持了优秀的推理速度，为系统的实时运行奠定了坚实基础。改进后的模型网络结构如图4.2所示，网络结构代码如代码4.1所示。

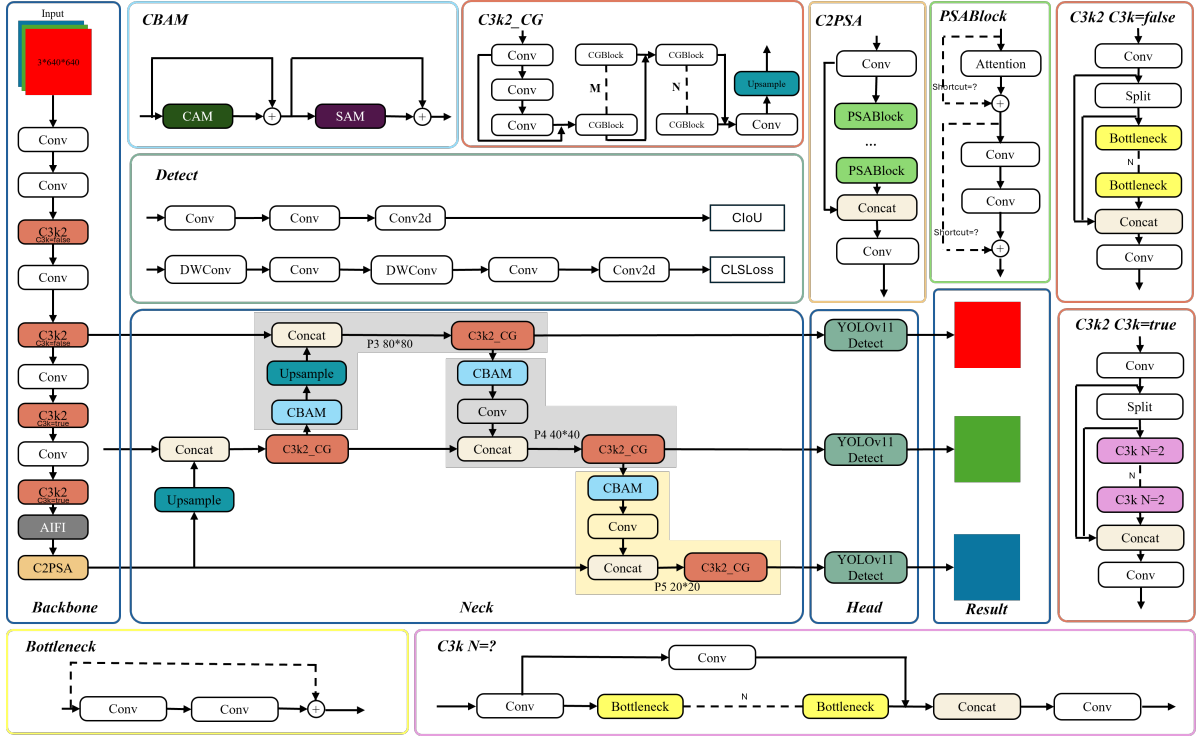


图 4.2 改进的 YOLOv11 网络结构图

代码 4.1: 网络结构.yaml

```

1 backbone:
2   - [-1, 1, Conv, [64, 3, 2]] # 0-P1/2
3   - [-1, 1, Conv, [128, 3, 2]] # 1-P2/4
4   - [-1, 2, C3k2, [256, False, 0.25]]
5   - [-1, 1, Conv, [256, 3, 2]] # 3-P3/8
6   - [-1, 2, C3k2, [512, False, 0.25]]
7   - [-1, 1, Conv, [512, 3, 2]] # 5-P4/16
8   - [-1, 2, C3k2, [512, True]]
9   - [-1, 1, Conv, [1024, 3, 2]] # 7-P5/32
10  - [-1, 2, C3k2, [1024, True]]
11  - [-1, 1, AIFI, [1024, 8]] # 9
12  - [-1, 2, C2PSA, [1024]] # 10
13
14 head:
15  - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
16  - [[-1, 6], 1, Concat, [1]] # cat backbone P4
17  - [-1, 2, C3k2_ContextGuidedBlock, [512, False]] # 13
18
19  - [-1, 1, CBAM, [512]]
20  - [-1, 1, nn.Upsample, [None, 2, "nearest"]]

```

```

21 - [[-1, 4], 1, Concat, [1]] # cat backbone P3
22 - [-1, 2, C3k2_ContextGuidedBlock, [256, False]] # 16 (P3/8-small)
23
24 - [-1, 1, CBAM, [256]]
25 - [-1, 1, Conv, [256, 3, 2]]
26 - [[-1, 13], 1, Concat, [1]] # cat head P4
27 - [-1, 2, C3k2_ContextGuidedBlock, [512, False]] # 19 (P4/16-medium)
28
29 - [-1, 1, CBAM, [512]]
30 - [-1, 1, Conv, [512, 3, 2]]
31 - [[-1, 10], 1, Concat, [1]] # cat head P5
32 - [-1, 2, C3k2_ContextGuidedBlock, [1024, True]] # 22 (P5/32-large)
33
34 - [[16, 19, 22], 1, Detect, [nc]] # Detect(P3, P4, P5)

```

在管道缺陷检测任务中，图像通常具有一些显著特点。首先，缺陷尺寸较小，如腐蚀、裂纹等常见缺陷在图像中仅占据很小的像素区域，容易被检测算法忽略。其次，背景较为复杂，管道内部环境恶劣，常伴随有污渍、水渍等多种干扰因素，增加了图像的噪声和干扰信息。此外，由于管道内部光照条件差，拍摄图像往往质量较低，这也进一步提高了缺陷检测的难度。

接下来，将在本文 4.2-4.4 章节分别讲述三个模块的结构以及它们在管道缺陷检测项目中的优势。

4.2 AIFI 模块设计与应用

4.2.1 模块优势

AIFI (Adjacent Interactive Feature Integration) 模块是一种专注于增强特征图中尺度内信息交互的结构。该模块最初被应用于目标检测模型 RT-DETR 中，并在提升模型对小目标以及复杂背景下的检测能力方面取得了显著效果。

YOLOv11 的基准架构将 SPPF 模块作为核心组件。然而，在处理高级图像特征时，SPPF 模块存在一定的局限性。具体来说，其在多尺度特征交互过程中，未能充分考虑高级特征与低级特征在语义信息上的显著差异，从而导致计算资源的浪费和计算冗余。

基于上述分析，使用 AIFI 模块进行了替换。它采用单尺度 Transformer 编码器对 S5 特征层进行内部信息处理，通过精简网络结构显著降低了相邻层级间的计算冗余。通过这种方式，AIFI 模块不仅降低了计算成本，还提高了模型的性能。同时，通过引入自注

意力机制，模型可有效捕获图像特征间的语义关联，从而增强目标定位与缺陷识别的检测精度。

与传统的 SPPF 模块相比，AIFI 模块在处理高级图像特征时展现出更高的效率和准确性。具体而言，AIFI 模块显著降低了延迟（快 35%），同时提高了准确性（AP 高 0.4%）^[9]。通过对高级特征层（例如 S5 层）引入自注意力机制，专注于同一尺度内的特征交互，能够精准捕捉图像中概念实体之间的关联，进而增强模型的定位和识别能力。

在本管道缺陷检测项目中，AIFI 模块展现出显著的优势。通过增强高级特征之间的联系，AIFI 模块能够更准确地识别出缺陷的特征和位置，提高对管道裂缝、腐蚀等缺陷的识别精度。同时，AIFI 模块通过减少不必要的特征交互，降低了计算成本，提高了检测速度，适合在实际工业场景中快速检测管道缺陷。此外，AIFI 模块能够更好地处理复杂背景下的图像特征，减少噪声和干扰对检测结果的影响，从而提高管道缺陷检测算法的鲁棒性。

4.2.2 模块构建

AIFI（Adjacent Interactive Feature Integration）模块是一种基于 Transformer 编码器的结构，旨在通过尺度内特征交互增强特征图的表达能力。该模块继承自 TransformerEncoderLayer 类，并对其进行了扩展以适应特定的特征交互需求。AIFI 模块的核心设计包括以下几个关键部分：

- 初始化方法：AIFI 模块在初始化时接收输入通道数 $c1$ 、中间层维度 cm 、注意力头数 num_heads 、dropout 概率、激活函数 act 以及是否在归一化前进行操作 $normalize_before$ 。这些参数为模块提供了灵活性，使其能够适应不同的特征维度和任务需求。
- 前向传播：在前向传播过程中，AIFI 模块首先将输入特征图 x 的形状从 $[B, C, H, W]$ 展平为 $[B, H \times W, C]$ ，以便进行 Transformer 操作。随后，模块调用 `build_2d_sincos_position_embedding` 方法生成二维正弦-余弦位置编码，并将其添加到输入特征中。位置编码的加入为 Transformer 提供了空间位置信息，有助于模型更好地理解特征图中不同位置的特征关系。
- 二维正弦-余弦位置编码：该方法首先创建一个二维网格，然后根据预定义的温度参数 $temperature$ 和嵌入维度 $embed_dim$ 计算相应的频率。最终，通过将网格坐标与

频率相乘并应用正弦和余弦函数，生成位置编码。位置编码的维度为 `embed_dim`，其设计确保了模型能够捕捉到特征图中不同地方的特征。

- 特征交互：通过维度重构将 Transformer 输出的特征转换为标准格式 $[B, C, H, W]$ ，以便与后续的网络结构兼容。AIFI 模块借助这种方式，不仅提升了特征图中尺度内信息的交互效率，还通过位置编码引入了空间位置信息，显著增强了模型对复杂背景和小目标的检测性能。

由于如今 YOLO 官方的库已经自带了 AIFI 模块，因此在设计模型时无需添加代码，可以在 yaml 文件修改模型结构时直接引用。根据图4.2改进的 YOLOv11 网络结构图，对 backbone 部分结构进行改进即可，如代码4.1所示。

4.3 C3K2_context_guided 模块设计与应用

4.3.1 模块优势

在目标检测领域，尤其是针对复杂场景的管道缺陷检测任务，传统 C3K2 模块存在一定的局限性。C3K2 模块主要依赖卷积操作来提取和融合特征，虽然能够捕捉局部特征，但在处理复杂场景时，仅依靠局部特征难以准确理解目标的语义信息。例如，在目标与背景或周围物体存在相似特征的情况下，C3K2 模块容易导致检测精度下降。此外，对于需要同时关注细节和整体结构的任务，C3K2 模块在多尺度特征融合方面不够高效，无法充分协调不同尺度特征之间的关系，从而影响检测效果。

为了解决这些问题，使用 C3K2_Context_Guided 模块进行了替换。此模块允许模型在局部和全局上下文之间建立联系，对于准确分类图像中的每个像素至关重要。通过这些提取器的协同作用，模型可以更好地理解目标在整体场景中的位置和语义关系，从而显著提高检测的准确性。此外，C3K2_Context_Guided 模块的另一个重要优势在于其优化的多尺度特征融合能力。基于改进型 C3K2 架构的多层级特征提取与融合机制，可有效提升模型对各种尺度的目标的适应性，同时在特征传递过程中显著降低信息损耗。这种能力在管道缺陷检测中尤为重要，因为管道缺陷可能存在于不同的尺度上，例如微小的裂纹、变形和较大的腐蚀区域。C3K2_Context_Guided 模块的多尺度特征融合能力使其能够同时检测到这些不同尺度的缺陷，提高了检测算法的全面性和可靠性。

在管道缺陷检测任务中，C3K2_Context_Guided 模块的优势尤为明显。管道自身和内部环境往往具有复杂的纹理和背景，可能存在光照不均匀、背景干扰等因素。在这种复杂环境下，C3K2_Context_Guided 模块能够通过丰富的上下文信息提取，更准确地识

别缺陷的边界和特征，更好地理解整个管道场景，从而提高检测精度。此外，由于管道缺陷的尺度差异较大，C3K2_Context_Guided 模块的多尺度特征融合能力使其能够同时检测到不同尺度的缺陷，进一步提升了检测算法的性能。

4.3.2 模块构建

C3k2_Context_Guided 模块是一种结合了上下文引导块（Context Guided Block, CG Block）的改进型 CSP Bottleneck 模块。CG Block 引入了局部特征提取器 f_{loc} 、周围上下文提取器 f_{sur} 、联合特征提取器 f_{joi} 以及全局上下文提取器 f_{glo} ，能够同时获取局部特征、周围上下文和全局上下文信息^[10]，并将这些信息进行融合，从而显著提升模型对复杂场景的理解能力。其结构图如图4.3所示。首先通过 1×1 卷积层实现通道压缩，随后通过并行的 f_{sur} 和 f_{sur} 分别提取细粒度特征与区域关联信息。融合后的多层级特征经由 f_{joi} 进行交互，最终通过 f_{glo} 进一步优化后输出。

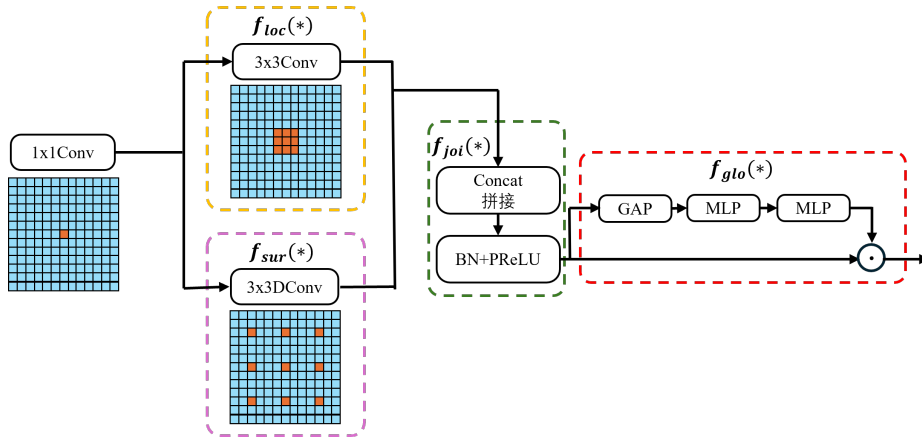


图 4.3 CG block 结构图

(1) 局部特征提取器 (f_{loc})

f_{loc} 的核心功能是从输入的特征图中提取特定局部区域的特征。它借助标准的 3×3 卷积层来实现，能够有效捕捉输入数据中的细节信息。这些提取出的局部特征后续将与周围的上下文特征相结合，从而形成对各个区域的完整理解。

(2) 周围上下文提取器 (f_{sur})

f_{sur} 通过空洞卷积（Dilated Convolution, DConv）来增大神经网络感受野，来有效提取更大范围的缺陷上下文特征。这种方法在保持计算效率的同时，显著提升了上下文信息捕获能力，而不仅仅是局部的细节。通过结合局部特征和周围上下文，模型可以更好地理解复杂的场景。

(3) 联合特征提取器 (f_{joi})

f_{joi} 的核心功能在于融合局部细节特征与相邻区域上下文信息。具体实现中，首先采用特征拼接方式将两类信息进行整合，形成联合特征表达，继而通过参数化线性整流函数与批量标准化处理，有效增强了特征的表征能力。BNPReLU 结合了批量归一化和参数化 ReLU 激活函数。这种组合能够有效增强特征表示的能力。

(4) 全局上下文提取器 (f_{glo})

f_{glo} 利用全局平均池化 (Global Average Pooling, GAP) 对整个特征图进行全局信息聚合，并通过一个两层的感知机 (Multilayer Perceptron, MLP) 进行后续处理。最终，通过缩放层将所提取的全局上下文信息与联合特征相融合，以此突出重要的特征部分并削弱不重要的特征部分。这种设计使得模型能够根据全局信息动态调整局部特征的重要性，从而提高检测的准确性。

(5) C3k2_Context_Guided 模块

C3k2_Context_Guided 是一个结合了上述所有组件的完整的模块。其结构图如图4.4所示。这种模块特别适用于高精度目标检测任务，如管道缺陷检测。

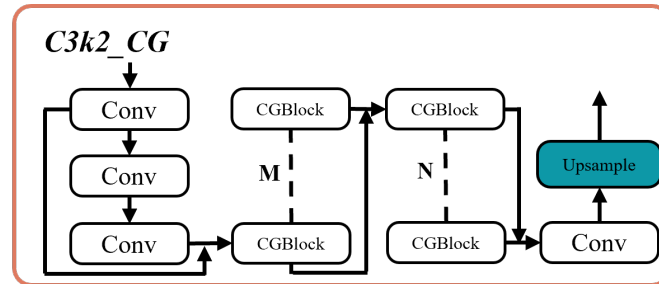


图 4.4 C3k2-CG 结构图

训练前，在 `ultralytics/nn/tasks.py` 文件中，我进行了必要的模块导入操作，并注册了 C3k2-CG 模块。接着，在 `parse_model` 函数中添加了对该模块的支持。最终，依据代码4.1中定义的结构，对模型的 `head` 部分网络架构进行了相应的修改，加入了 C3k2_ContextGuided 模块。

4.4 CBAM 模块设计与应用

CBAM 作为一种典型的双路注意力机制，通过整合通道域与空间域的互补信息，可自适应地优化特征响应。将 CBAM 模块嵌入 P3、P4、P5 特征层后（如代码4.1所示），模型能够更精准地聚焦关键特征区域，同时削弱无关特征的影响，从而提升特征的区分度与鲁棒性。

针对 YOLOv11 网络的多尺度检测特性，其头部结构中 P3-P5 特征层分别负责不同

尺度目标的识别。引入 CBAM 注意力机制能有效优化各层级特征表达能力：浅层特征 P3 通过该模块强化了小目标细节捕捉能力，而深层特征 P5 则增强了全局特征的解析效果。该改进方案在提升检测精度的同时，未显著增加模型复杂度，其轻量化特性使网络在保持实时推理速度方面具有优势，故适用于实际工程场景中的管道缺陷识别任务。

5 实验与模型性能评估

5.1 训练过程与参数设置

为了验证本研究所提出的管道缺陷检测模型的有效性，本文基于第三章构建的高质量数据集，对改进后的 YOLOv11 模型进行了系统性的训练与评估。本节将详细介绍训练过程的整体流程以及关键参数设置。

模型训练和测试的配置环境如表5.1所示。

表 5.1 模型训练时的软硬件环境

模型参数	值
处理器	AMD Ryzen 9 7940H w/ Radeon 780M Graphics @ 4.00 GHz
训练环境	CUDA12.0
操作系统	Windows10
开发环境	python3.9,Pytorch 2.0.0,Torchvision 0.15.0

在训练过程中，数据集被划分为训练集、验证集和测试集，具体数量见表3.1。图像输入尺寸统一为 640×640 ，批量大小设为 32，以平衡模型精度和推理速度。表5.2展示了模型的基本训练参数。

表 5.2 模型参数设置

模型参数	值
训练轮数	200
每个批次中的图像数量	32
SGD 优化器的动量	0.937
学习率	自适应
使用的优化器	SGD
输入图像的尺寸	640
余弦学习率调度器	是
自动混合精度 (AMP) 训练	是
Mosaic 数据增强	是
在最后第几个轮次禁用 mosaic 增强	10
交并比	0.5

为了增强模型的鲁棒性和泛化能力，避免数据的共同特征对模型权重产生负面影响，在模型训练阶段，随机旋转、水平翻转与 Mosaic 数据增强等技术被综合运用，以提升算法的泛化能力。这些增强策略能够模拟真实应用环境中不同角度、光照和干扰下

的图像特征，从而提升模型在复杂场景中的表现能力。

本节的训练过程与参数设置旨在确保模型在精度与效率之间取得良好平衡，为后续的性能评估与对比实验提供了坚实基础。

5.2 性能评价指标与测试结果

为综合评估改进型 YOLOv11 算法在管道缺陷识别任务中的性能，实验选取精确率 (Precision)、召回率 (Recall) 和平均精度均值 (mAP) 三项目标检测通用评估指标，这些指标可通过以下公式进行计算。

(1) 精确率 (Precision)

$$\text{Precision} = \frac{TP}{TP + FP} \quad (5-1)$$

其中，真正例 (TP) 指实际为正类且被正确识别的样本数，假正例 (FP) 则为实际为负类但被误判为正类的数量。精确率为检测到的目标中正确识别的比例。

(2) 召回率 (Recall)

$$\text{Recall} = \frac{TP}{TP + FN} \quad (5-2)$$

其中，假负例 (FN) 则对应真实为正类但被误判为负类的样本数。召回率体现了模型检测出所有真实目标的能力。

(3) 平均精度 (AP)

$$\text{mAP@0.5} = \sum_{i=0}^N \int_0^1 P_i(R_i) d(R_i) \quad (\text{IoU}_{\text{thresh}} = 0.5) \quad (5-3)$$

平均精度 (AP) 通过计算不同召回率对应的精确度均值来评估模型性能。作为目标检测任务的关键指标，平均精度均值 (mAP) 整合多阈值下的检测结果，全面衡量算法稳定性。其中 mAP@0.5 代表交并比 (IoU) 判定标准为 0.5 时的综合评估值，是衡量基础检测能力的重要参数。

本实验在测试集上进行了充分测试，在训练轮数设置为 200 轮的条件下，统计了模型在四类典型缺陷（腐蚀、裂缝、堵塞、变形）上的检测性能，如表5.3所示。实验结果显示，模型在各类缺陷的检测中均表现出较高的精度与稳定性。

从表中可以看出，不同缺陷的识别并不同样简单。模型在裂缝和堵塞这两类缺陷上

表 5.3 模型在测试集 (训练轮数=200) 的评价结果

类别	目标数量	精确率	召回率	mAP@0.5
全部	144	0.874	0.832	0.855
腐蚀	96	0.843	0.811	0.838
变形	5	0.678	0.709	0.712
堵塞	28	0.926	0.871	0.995
裂缝	15	0.949	0.837	0.875

的 mAP@0.5 指标较高，分别达到了 0.875 和 0.995，说明模型在识别这类特征明显的缺陷时具有较强的判别能力。而在腐蚀和变形类别上，该模型表现出相对较差的验证性能。这可归因于样本量小，且缺陷较复杂，使得识别更加困难。虽然存在一定的检测难度，但精确率和召回率仍保持在较优水平，表明模型在细粒度特征提取方面的能力得到了有效提升。这得益于本文引入的 CBAM 注意力机制、AIFI 交互式特征融合模块以及 C3K2_context_guided 上下文感知模块在特征提取和多尺度融合方面的优化作用。

总体来看，本模型在多个缺陷类型上的检测精度均达到了实际应用的可接受范围，验证了改进方法的有效性。后续将进一步在更大规模数据集上进行测试，并尝试部署于实际环境中，以评估其实时检测与鲁棒性表现。

5.3 消融实验结果

为分析各优化模块对模型性能的有效性，通过分阶段消融实验系统评估了不同组件的实际作用。实验分别移除 AIFI 模块、C3K2_context_guided（简称 C3K2_CG）模块和 CBAM 模块，并组合形成多个变体模型，在相同训练参数（训练轮数=200）、验证集和评估指标下进行对比分析。

表 5.4 消融实验结果

	AIFI	C3K2_CG	CBAM	精确率	召回率	mAP@0.5
1	√	√	√	0.874	0.832	0.855
2	√	√	×	0.869	0.824	0.849
3	√	×	√	0.861	0.818	0.838
4	×	√	√	0.864	0.820	0.842
5	√	×	×	0.857	0.814	0.830
6	×	√	×	0.859	0.816	0.835
7	×	×	√	0.858	0.815	0.831
8	×	×	×	0.857	0.813	0.826

表5.4展示了不同实验组的模块配置及其验证集评估指标。数据分析显示，当三个

功能模块同时启用时（实验组 1），系统呈现出最优性能表现，在精确率、召回率以及 $mAP@0.5$ 指标上均取得了最高分数，说明各模块在协同作用下显著增强了模型的特征提取与检测能力。

当移除任意一个模块（实验组 2 至实验组 4）和移除任意两个模块（实验组 5 至实验组 7）时，模型性能均有不同程度的下降。其中，去除 C3K2_CG 模块（实验组 3、5、7）导致 $mAP@0.5$ 降幅最为明显，表明该模块在补充上下文信息、提升模型感知能力方面发挥了关键作用。CBAM 模块的去除同样带来了明显影响，说明注意力机制对于提升目标检测精度尤为重要。而 AIFI 模块的贡献则更多体现在提升模型对多尺度目标的识别能力，特别是在复杂背景下保持较高召回率。

在未引入任何改进模块的基础模型（实验组 8）中，模型性能最低，验证了本文所提出模块的有效性与必要性。

因此，AIFI、C3K2_CG 及 CBAM 三个模块各具优势，在特征融合、上下文建模及注意力引导方面互为补充，三者结合能够显著提升管道缺陷检测模型的整体性能。

5.4 对比实验与分析

为评估改进 YOLOv11 模型在管道缺陷识别中的性能表现，本研究选取多种主流检测模型进行对比实验，包括 YOLOv3、YOLOv5s、原始的 YOLOv11、Faster-RCNN 以及 CenterNet。各模型在相同的数据集、训练轮数（200 轮）与评估指标条件下进行训练和测试，性能结果如表 5.5 所示。

表 5.5 对比实验结果

模型	精确率	召回率	$mAP@0.5$
改进的 YOLOv11	0.874	0.832	0.855
YOLOv3 ^[35]	0.787	0.737	0.798
YOLOv5s ^[36]	0.806	0.760	0.776
YOLOv11	0.857	0.813	0.826
Faster-RCNN ^[37]	0.523	0.588	0.788
CenterNet ^[38]	0.621	0.573	0.739

从表中可以看出，改进后的 YOLOv11 模型在精确率、召回率以及 $mAP@0.5$ 三个指标上均取得了最优成绩，表明所引入的 AIFI、C3K2_CG 和 CBAM 模块在增强模型特征表达能力、感受野扩展以及目标注意力引导方面起到了积极作用。

相比之下，YOLOv3 和 YOLOv5s 在复杂工业场景中仍存在改进空间，特别是对微小缺陷及不规则形态目标的识别能力较弱，常出现目标漏判现象，其查全率与定位精度

仍需进一步优化。Faster-RCNN 作为双阶段检测器，在精确率方面表现尚可，但在实时性与轻量化方面存在劣势，且对复杂背景的适应能力较弱。CenterNet 模型虽然能实现端到端回归检测，但在处理多尺度目标时效果一般，容易出现目标错检现象。

此外，改进 YOLOv11 相比原始 YOLOv11 有所提升，说明本文的网络结构优化方案有效增强了模型的检测能力与稳定性，尤其是在实际管道缺陷多样、形态复杂的情况下，改进模型能更准确地识别和定位各类缺陷。

综上所述，本文所提出的改进的 YOLOv11 模型在保持较高检测精度的同时，兼顾了检测速度与模型鲁棒性，具备良好的实用性与推广前景，能够更好地满足智能化管道缺陷检测系统的实际应用需求。

6 系统设计与实现

6.1 系统分析

6.1.1 应用背景分析

随着城市化进程的加速，城市管道系统作为城市基础设施的核心组成部分，其运行的安全性与可靠性受到越来越多的关注。然而，传统的管道检测方式有效率低、覆盖范围有限、响应不及时等问题，满足不了城市管网高效运维的实际要求。

近年来，人工智能技术，尤其是图像识别技术的飞速发展，为管道系统的智能化管理提供了新的思路和解决方案。通过在管线上安装高清无线摄像头，结合深度学习算法，可以实现对管道系统状态的实时监测和异常情况的自动识别。基于深度学习的管道缺陷检测系统能够自动识别管道缺陷类型和等级，并根据需要生成详细的检测报告，极大地提升了检测效率和准确性。这种技术的应用能够及时发现潜在问题，避免进一步的损失和问题扩大化。

此外，集成水位传感装置的系统可实现液位动态监测，提供准确的预警，辅助城市防汛抗旱工作。这种实时监测和预警能力显著提升了管道系统的安全性和可靠性，为城市基础设施的现代化管理提供了有力支持。

6.1.2 市场需求分析

从市场需求来看，市政领域对管道检测的需求最为突出，供水、排水、燃气管道的检测修复需求占比超过 70%。2024 年，城市供水管网漏损率超过 15%，老旧管道更新需求迫切。此外，工业领域如石油、化工、电力等行业对管道检测修复的需求也较为稳定，占总需求的 25%。2024 年化工管道检测市场规模同比增长 30%，显示出该领域的强劲需求^[39]。

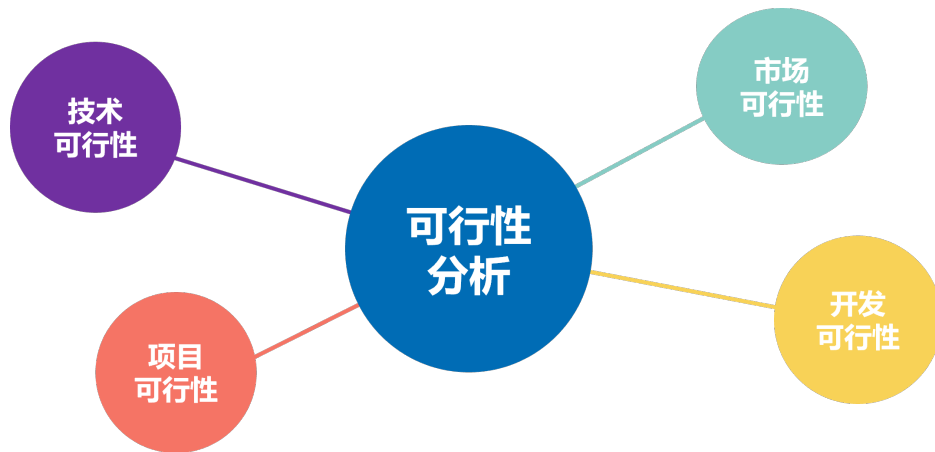
在技术层面，智能化、自动化检测技术成为市场的主要需求方向。随着人工智能、大数据分析和物联网技术的发展，市场对高精度、多功能化以及远程控制的检测设备需求不断增加。

政策方面，国家及地方政府陆续出台相关政策，鼓励和推动管道检测技术的升级与应用。这些政策不仅促进了技术创新，还为市场扩张提供了有力支持。例如，政府对老旧管网改造的投入加大，推动了市政管道检测市场的增长^[40]。

总体来看，管道缺陷检测系统在市政和工业领域都拥有广阔的市场需求。智能化、自动化检测技术的不断发展以及政策的有力支持，为该行业的持续增长奠定了坚实基础。展望未来，随着技术的持续进步和市场需求的进一步拓展，管道检测系统将在更多领域大显身手，为城市基础设施的安全运行和可持续发展提供关键支撑。

6.1.3 可行性分析

本文将可行性分析分为技术可行性、市场可行性、项目可行性和开发可行性四个部分，如图6.1所示，从不同角度对基于深度学习的管道缺陷检测系统进行全面评估，以确保项目的顺利实施和可持续发展。



(1) 技术可行性

在系统开发过程中，本人具备扎实的计算机专业知识基础，这为系统的搭建与实现提供了良好的基础条件。通过长期的学习和实践，积累了丰富的经验，尤其是在目标检测关键技术（如 YOLO 技术）领域，进行了深入的研究和实践。这些经验为数据挖掘与处理、系统设计、模型训练、算法优化和测试等一系列工作提供了重要保障，减少了开发过程中可能出现的技术问题。

在开发环境方面，本研究基于 Windows 10 操作系统构建开发环境，部署了包括 Python 3.9、Java8 和 HTML5 等编程语言，集成 IntelliJ IDEA 与 VSCode 作为代码编辑器，采用 Flask 框架实现后端功能，结合 Bootstrap 和 jQuery 进行前端开发，并配置了相关调试工具。这些工具和环境为系统的开发提供了强大的技术支持。此外，所在学校具备高速互联网连接和大量主流数据库，能够轻松查阅 ACM、SCI、EI、Elsevier、知网、万方等学术资源，为系统的理论研究和探索提供了丰富的参考。同时，利用 GitHub 等开源平台上的资源，作为技术参考，进一步支持了系统的开发工作。Python 编程环境

集成了 Numpy、Pandas 等高效工具集，结合计算机视觉与深度学习框架，为系统设计与功能实现奠定了技术基础。

(2) 市场可行性

随着城市化进程的不断推进，城市管道系统的建设和管理变得越来越重要。监控系统能够提供实时数据和精细化管理，满足城市决策者对城市基础设施状态的了解和优化的需求。同时，公众对环境保护和可持续发展的关注度也在增加，对于管道系统的监控和管理要求逐渐提高。随着信息技术的迅猛发展，传感装置、数据智能分析以及远程传输与监控等关键技术日趋完善，并且逐渐降低了成本，这为城市管线系统监控系统的实施提供了可行的技术支持。

政府对于城市基础设施建设和环境保护的重视程度逐渐提高，相关法律法规也在逐步完善。政府对于城市管道系统监控系统的推广和应用提供了政策支持，为市场的发展提供了有利条件。全球范围内，许多城市都面临着管道系统的管理和优化需求，尤其是快速城市化地区。随着城市规模的扩大和城市治理水平的提高，市场规模将不断增长，为管道缺陷检测系统提供了广阔的市场空间。

(3) 项目可行性

在这一部分，从经济、社会、政策三个方面出发，对项目进行可行性分析。

经济方面，管道缺陷检测系统具有显著的经济价值。通过智能化的检测手段，能够有效降低人工检测的成本和时间，提高检测效率和准确性。减少因管道缺陷导致的资源浪费和维修成本，为大量行业尤其是水务行业带来可观的经济效益。

社会方面，管道缺陷检测系统的应用能够显著提升管道行业的运行安全和管理效率。利用智能图像识别技术实时监测管线的状态和变化，实现对管道缺陷的精准识别和定位，为城市的建设和管道管理提供强有力的支持。该系统能够有效降低误判率和漏报率，提高识别效率，简化数据整理和报告撰写的工作量，提供辅助决策支持，帮助管理人员更好地制定维修计划和资源调配。

政策方面，随着环境保护和城市管理的重视，未来可能会有相关政策出台以促进管道监控和管理的发展。本项目顺应中国智造的潮流和趋势，具备实施可行性。

(4) 开发可行性

演进式迭代是软件工程中的重要概念，其核心理念是通过逐步迭代的方式不断改进和完善系统。在迭代式开发过程中，系统建设被分解为若干阶段，每个阶段聚焦特定功能开发。各阶段按照“需求分析-方案设计-编码实现-测试验证”的流程推进，最终形成

具备基础功能的子系统版本。这不仅有助于及时满足用户反馈的需求，还能够快速适应不断变化的技术和设施环境。

简洁设计原则强调在系统开发中遵循简洁、清晰和易维护的设计原则。根据现有的需求进行建模，日后需求有变更时，再进行系统重构，以提高代码质量和系统可维护性。需求管理着重在整个开发周期中灵活管理和同步变化的需求，确保系统持续适应用户、市场和环境的变化。任务分解优化将智能管道缺陷检测系统拆分成多个小模块进行开发，例如实时视频传输、目标检测和 PDF 生成等，降低了开发风险，也减少了后期调试的工作量，确保了项目的可行性和开发的可控性。

6.1.4 功能需求分析

(1) 实时监控功能需求分析

实时监控功能是管道缺陷检测系统的核心组成部分，旨在为用户提供管道内部的实时视频流，以便及时发现和处理潜在的缺陷。以下是该功能的具体需求分析：

- 实时视频流显示：系统应具备高清晰度的实时视频流显示功能，确保用户能够清晰地观察管道内部的情况。视频流应无明显延迟，以满足实时监控的需求。
- 自动缺陷检测与标注：系统应集成自动缺陷检测算法，能够在实时视频流中自动识别管道缺陷，并在视频画面上进行标注，方便用户快速定位问题区域。
- 用户界面友好性：监控界面应设计简洁直观，包含实时视频流检测区域和摄像头切换面板等关键信息展示区域，确保用户能够快速获取所需信息。

(2) 设备管理功能需求分析

设备管理功能是确保系统稳定运行和有效监控的关键，以下是该功能的具体需求分析：

- 摄像头信息展示：系统应提供摄像头列表界面，展示摄像头的信息。
- 摄像头切换：用户应能够通过系统进行摄像头切换，以适应不同的检测需求。
- 摄像头兼容性：系统应支持多种类型的摄像头接入，确保与市场上主流的高清摄像头和特种摄像头的兼容性，提高系统的通用性和灵活性。

(3) 报警信息管理功能需求分析

作为管线缺陷监测的重要模块，报警信息管理功能需满足以下基本要求：

- 信息记录：当系统检测到管道缺陷时，应自动记录缺陷信息，包括检测时间、缺陷类型等信息。
- 信息处理：用户应能够对信息进行查看与处理，对已处理的报警信息可以进行删除操作。
- 报告导出：系统应支持将报警信息导出为 PDF 报告格式，方便用户进行存档。报告应包含检测时间、检测结果、缺陷照片等关键信息。

(4) 场站管理功能需求分析

场站管理功能是系统对管道检测项目的组织和管理的关键，以下是该功能的具体需求分析：

- 场站信息管理：系统应允许查看场站信息，包括场站名称、位置等详细信息，实现对场站的全面管理。
- 场站地理信息标注：系统应支持在地图上标注场站的位置信息，提供直观的地理分布展示，方便用户了解场站的地理位置和相互关系。

6.2 系统设计

6.2.1 技术路线设计

本系统技术路线构成如图6.2所示。

6.2.2 功能模块设计与实现

(1) 实时监控功能

本系统的核心功能在于实现实时监控能力。系统能实时传输管道内部的视频流，使用户在监控界面上即时查看管道的运行状态。与此同时，系统集成了自动检测模块，在发现管道内部存在缺陷时，能立即触发报警机制，提醒用户尽快处理相关问题，以保障管道系统的正常运行和维护效率。

数据采集作为构建可靠和可持续解决方案的基础环节，在整个项目中扮演着至关重要的角色。有效的数据采集过程能够洞察系统状态、捕捉运行趋势，同时也有助于发掘潜在的问题与改进空间，为项目的持续优化和迭代发展打下坚实基础。因此，本项目首先确立了数据采集的目标与方式。

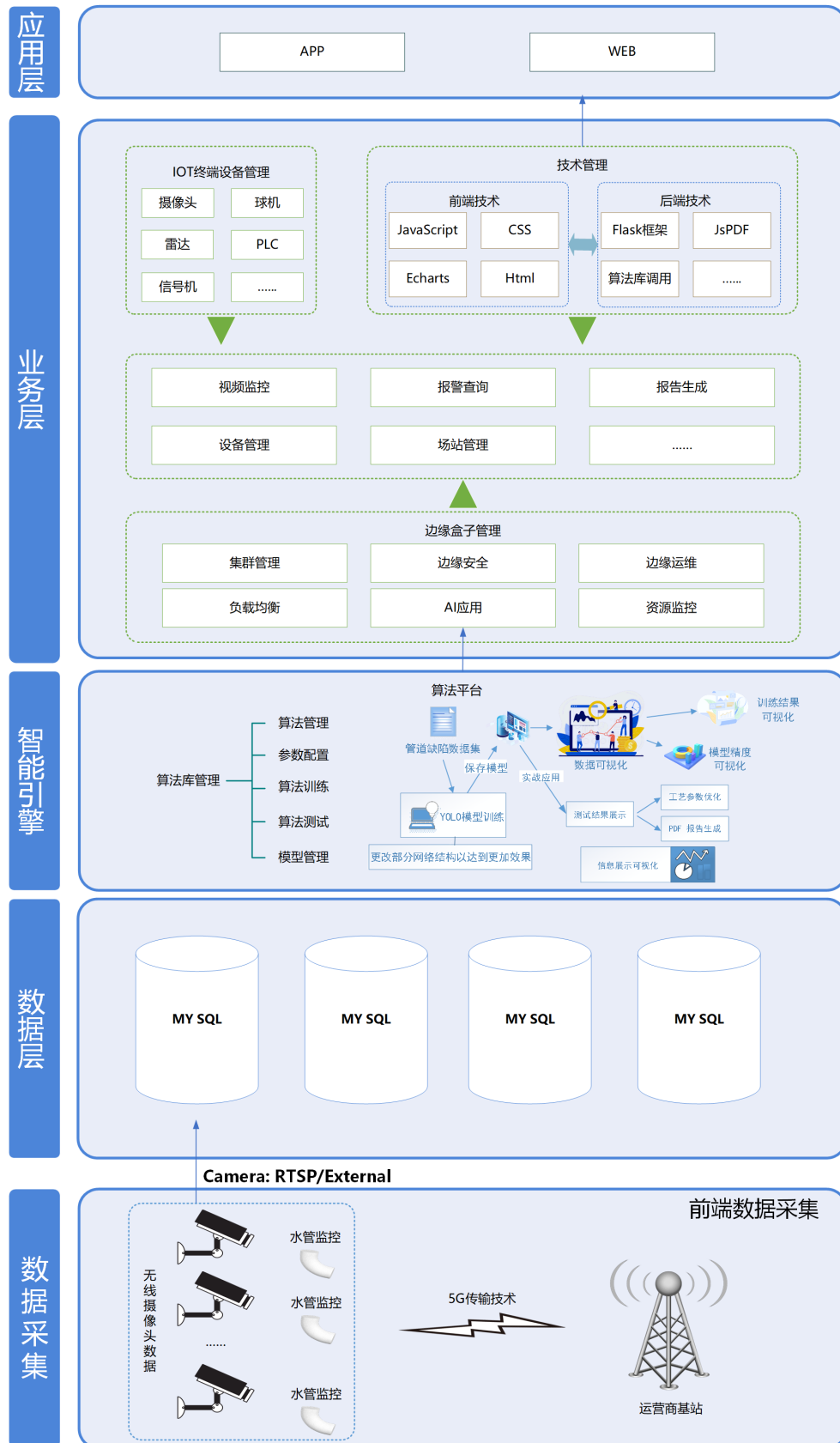


图 6.2 技术路线图

在采集目标方面，首先明确了数据源可以来自于无线高清摄像头，并建立了相应的数据获取机制。其次，系统定义了所需采集数据的类型与格式。再次，合理设定了采集频率与数据量，确保数据既具有时效性，又能满足处理要求。同时，还设立了数据质量控制标准，确保所采集数据在完整性、清晰度与同步性等方面达到预期标准。

在采集方式方面，系统采用可在管道内规律移动的高清摄像头进行图像采集。实际应用中，视频传输过程中容易出现掉帧、卡顿、延迟和画面模糊等问题，这些问题会直接影响企业用户的观看体验及系统的缺陷检测准确率。为解决这一问题，项目引入了 RTSP（Real-Time Streaming Protocol）实时视频传输协议理论，形成高效可靠的传输方案。

当前，管线监控系统常用的数据传输方式包括普通网线、光纤、无线网桥、3G/4G 网络和微波传输等。在这些方式中，5G 技术凭借其高带宽、低延迟、广连接等特性，尤其适用于实时视频和多类传感数据并发传输的场景。虽然本项目未直接采用 5G 技术，但对 5G 理论的研究为后续技术升级提供了理论基础。RTSP 协议则具备低延迟、高稳定性与良好的跨平台兼容能力，在监控、视频会议及在线直播等领域广泛应用，已成为实时视频流传输的首选方案。

为了支持从 RTSP 流中读取视频数据并进行目标检测，我设计了如下的处理逻辑。核心代码如代码6.1所示。首先，通过判断输入的源地址是否以 rtsp:// 开头，确定输入是否为 RTSP 流。对于 RTSP 流，使用 LoadStreams 方法加载视频流，并将其帧数据传递给目标检测模型进行推理。检测到的目标信息（包括类别和置信度）会被标注在视频帧上，并通过 OpenCV 的 imencode 方法转换为字节流，以便存储到数据库中。最终，检测结果（包括摄像头编号/RTSP 地址、检测时间、检测结果和图片）会被存入数据库，以便后续的查询和分析。

代码 6.1: RTSP.py

```
1  # camera.py
2  # 判断视频源是否为文件、URL 或摄像头
3  is_file = Path(Camera.video_source).suffix[1:] in (IMG_FORMATS + VID_FORMATS)
4  is_url = Camera.video_source.lower().startswith(('rtsp://', 'rtmp://'))
5  webcam = Camera.video_source.isnumeric() or Camera.video_source.endswith('.txt') or (is_url and not
        is_file)
6  # 根据视频源类型选择加载数据的方式
7  if webcam: # 输入是普通摄像头或者网络摄像头
8      dataset = LoadStreams(Camera.video_source, img_size=imgsz)
9  else:
```

```

10     dataset = LoadImages(Camera.video_source, img_size=imsize)
11
12 # app.py
13 if source_name.lower().startswith(('rtsp://', 'rtmp://')): # 如果用户输入了RTSP地址, 则使用该地址
14     camera_instance = Camera(source_name, False) # 使用该地址创建 Camera 实例
15 elif source_name == "0": # 如果输入的是 "0", 则使用本地摄像头
16     camera_instance = Camera("0", False)
17 elif source_name == "1": # 如果输入的是 "1", 则使用外接摄像头
18     camera_instance = Camera("1", False)
19 else: # 如果输入的地址无效, 返回错误提示
20     return "Invalid source address", 400

```

通过结合 5G 与 RTSP 技术, 系统不仅可以实现高清视频的高效实时传输, 还可以显著降低数据传输过程中的延迟与丢包率, 进一步提升了系统的整体稳定性和用户体验。为实现该方案, 我参考相关资料, 了解到基于如下步骤可以在理论上完成关键技术的整合:

- **Step1: 建立 5G 连接:** 通过 5G 网络与所需的数据源或服务器建立连接。这可能需要特定的硬件设备或 5G 模块, 并使用相应的协议和认证方式建立通信。
- **Step2: 初始化 RTSP 流媒体:** 使用 RTSP (实时流传输协议) 建立与视频流媒体源的连接。这可以通过指定视频源的 URL 或其他标识符来实现。
- **Step3: 数据接收与处理:** 在一个循环中, 持续从 5G 网络接收数据和从 RTSP 流中获取实时视频数据。这可能涉及数据包的接收、解码和处理。例如, 从 5G 网络获取传感器数据或其他信息, 并从 RTSP 流中获取视频帧。
- **Step4: 数据处理与显示:** 对接收到的数据进行必要的处理, 如解析、验证、解码或其他操作。同时, 将处理后的视频数据在屏幕上显示或通过其他方式呈现出来。
- **Step5: 循环持续运行:** 在持续接收和处理数据的同时, 保持 5G 连接和 RTSP 流媒体通道处于活动状态, 确保实时性和持续性。

以下给出 5G 和 RTSP 结合算法的伪代码:

Algorithm 1 5G and RTSP Integration

- 1: **Input:** 5G connection, RTSP stream
- 2: Initialize 5G_connection


```

3: Initialize RTSP_stream
4: while true do
5:   data_5G  $\leftarrow$  receive_data_from_5G()
6:   video_RTSP  $\leftarrow$  receive_video_from_RTSP()
7:   process_data(data_5G)
8:   display_video(video_RTSP)
9: end while

```

综上所述，系统可以通过先进的数据采集机制实现稳定、高效、响应迅速的数据传输，为管道缺陷的及时发现与处理提供了强有力的技术保障。

接着，在目标检测方面，我采用了深度学习算法结合 Flask 后端的方式。整体流程可简要概括为：摄像头采集 → 模型检测 → 前端展示（同时存入数据库），如图6.3所示。

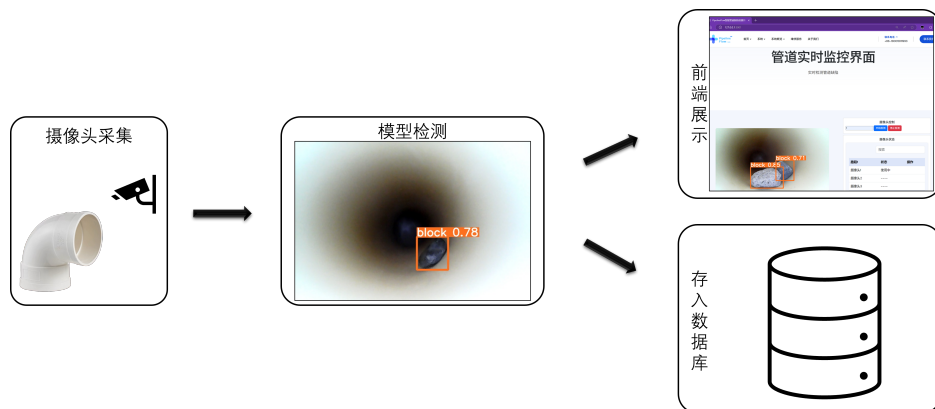


图 6.3 整体流程图

首先，系统调用摄像头，实现图像的实时采集。摄像头每次拍摄一帧图像后，会将该图像传输至后端的检测模块，由部署好的深度学习模型对图像进行缺陷识别。模型完成推理后，输出包含目标边框与类别信息的检测图像。随后，处理完成的图像结果由后端的主控制脚本进行统一调度与管理，并将结果推送至前端界面展示。最后用户可在浏览器中实时查看缺陷检测结果，包括检测框、类别标签以及置信度等信息。具体实现过程如图6.4所示。

在具体实现上，用户在前端页面的表单中输入摄像头索引或 RTSP 地址，并点击“开始检测”按钮。此时，JavaScript 脚本会阻止表单的默认提交行为，获取用户填写的视频源地址，并动态构造视频流的 URL，将其赋值给前端 `<img>` 标签的 `src` 属性，从而发起视频流的展示请求。前端请求到达后端后，由 Flask 框架定义的 `/video_feed` 路由负责接收视频源参数，并根据其类型（本地或 RTSP 地址）初始化 `Camera` 类实例，调用视

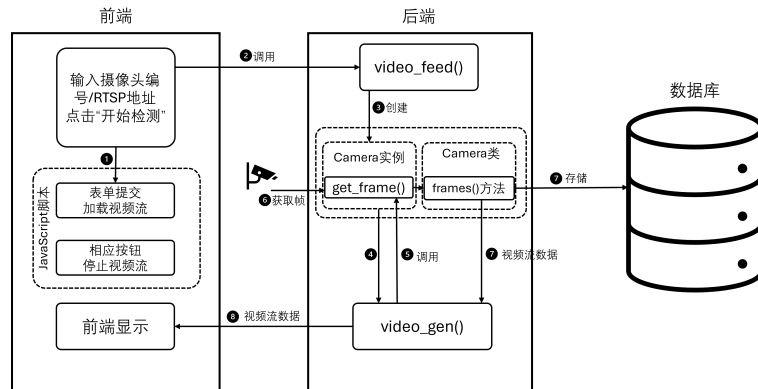


图 6.4 实时监控流程

频流生成函数 `video_gen()` 返回处理后的视频帧流数据。在 `video_gen()` 函数中，系统不断调用 `Camera` 实例的 `get_frame()` 方法获取每一帧图像。`Camera` 类中的 `frames()` 方法负责生成每一帧图像数据，并调用深度学习模型进行目标检测。检测模块会在每一帧图像中执行目标识别，并在识别结果中添加可视化标注，同时将检测时间、摄像头编号、识别标签、图片结果等信息写入数据库，如代码6.2所示。最终，每一帧经过编码后以字节流的形式持续返回前端页面。

代码 6.2: 检测与数据库存储逻辑.py

```

1  # camera.py
2  # 遍历每一张图像的检测结果
3  for i, det in enumerate(pred):
4      if webcam: # 如果是摄像头流，获取路径、图像和帧号
5          p, im0, frame = path[i], im0s[i].copy(), dataset.count
6      else: # 如果是视频文件，获取路径、图像和帧号
7          p, im0, frame = path, im0s.copy(), getattr(dataset, 'frame', 0)
8      p = Path(p)
9      save_path = str(out / p.name) # 定义保存路径
10     gn = torch.tensor(im0.shape)[[1, 0, 1, 0]] # 获取图像的尺寸
11     annotator = Annotator(im0, line_width=3, example= str(names)) # 创建注释器
12     if det is not None and len(det): # 如果有检测结果
13         det[:, :4] = scale_coords(img.shape[2:], det[:, :4], im0.shape).
14             round() # 将检测框坐标转换为原始图像尺寸
15         for *xyxy, conf, cls in det: # 遍历每个检测结果
16             label = '%s %.2f' % (names[ int(cls)], conf) # 获取类别名称和置信度
17             if conf > 0.35: # 只处理置信度大于35%的检测结果
18                 annotator.box_label(xyxy, label, color=colors( int(cls), True))
19
20     # 根据输入的摄像头索引设置摄像头名称

```

```

20     camera_name = "摄像头0" if Camera.video_source == "0" else "摄像头1" if Camera.
        video_source == "1" else Camera.video_source
21
22     # 将检测结果存入数据库
23     nowTime = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime()) # 当前时间
24     image = cv2.imencode('.jpg', im0)[1].tobytes() # 将图像转换为字节流
25     # 将检测结果存入数据库
26     obj = SQLHelper.fetch_one(
27         "INSERT INTO t_detect (camera, time, result, picture) VALUES (%s, %s, %s, %s)",
28         args=(camera_name, nowTime, label, image)
29     )

```

该流程实现了前后端协同、实时响应的管道缺陷检测系统，有效提升了检测的智能化与可视化水平。

(2) 摄像头管理功能

摄像头管理功能是本系统的重要组成部分，它允许用户对连接的摄像头进行集中管理。该功能为用户提供了一个直观的界面，用户可以通过该界面轻松查看当前连接的摄像头状态。此外，系统支持本地摄像头和网络摄像头（通过 RTSP 协议）的接入，确保了系统的兼容性和灵活性。用户可以在监控界面根据需要输入摄像头编号或 RTSP 地址进行摄像头的切换。点击“开始检测”按钮后，系统会根据用户输入的信息，动态加载对应的视频流并进行显示。

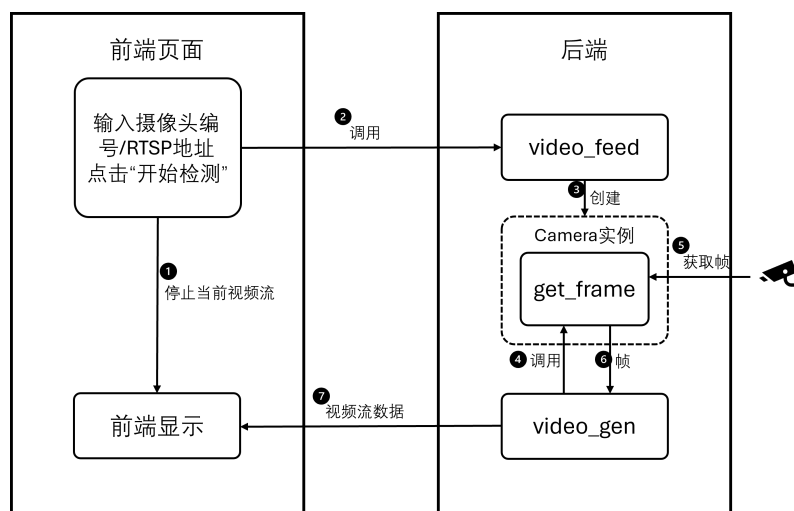


图 6.5 摄像头切换原理

这一过程通过前端页面与后端服务的紧密配合来实现，实现原理如图6.5所示。在后端，我们通过 Flask 框架定义了/video_feed 路由来处理视频流的加载和显示。当用户

在监控界面输入摄像头编号或 RTSP 地址并提交请求时，后端会根据输入的地址类型创建对应的 Camera 实例。

如代码6.3所示，video_feed() 函数首先获取用户输入的摄像头编号或 RTSP 地址，然后根据地址类型创建 Camera 实例。针对不同输入源类型，系统采用差异化处理机制：检测到 RTSP 协议地址时自动生成网络摄像头实例对象，而数字编号输入（如 0、1）则会创建本地摄像头实例。创建实例后，系统调用 video_gen() 函数，video_gen() 函数通过调用 Camera 实例的 get_frame() 方法获取每一帧图像，并将其封装为视频流数据返回给前端。

代码 6.3: 视频流数据生成并返回.py

```

1  # app.py
2  # 定义视频流生成器函数
3  def video_gen(camera):
4      """Video streaming generator function."""
5      global stop_detection
6      while not stop_detection: # 当 stop_detection 为 False 时，持续生成视频帧
7          frame = camera.get_frame() # 调用 Camera 类的 get_frame 方法获取视频帧
8          if frame: # 如果获取到帧，按照特定格式生成视频流数据
9              yield (b'--frame\r\n'
10                     b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')
11             time.sleep(0.5) # 每0.5秒检测一次，可以根据需要调整时间间隔
12
13 # 定义视频流路由，用于提供视频流数据
14 @app.route('/video_feed')
15 def video_feed():
16     """Video streaming route. Put this in the src attribute of an img tag."""
17     global stop_detection, camera_instance
18     stop_detection = False # 重置停止标志
19     source_name = request.args.get('source', default=None, type=
20         str) # 从请求参数中获取用户输入的视频源地址，默认为 None
21     print(f"Received source: {source_name}") # 打印接收到的地址
22
23     # 判断用户输入的地址类型
24     if source_name is None:
25         # 如果没有输入地址，返回空白响应
26         return "", 204
27
28     # 如果用户输入了RTSP地址，则使用该地址
29     if source_name.lower().startswith(('rtsp://', 'rtmp://')):

```

```

29     camera_instance = Camera(source_name, False) # 使用该地址创建 Camera 实例
30 elif source_name == "0": # 如果输入的是 "0", 则使用本地摄像头
31     camera_instance = Camera("0", False)
32 elif source_name == "1": # 如果输入的是 "1", 则使用外接摄像头
33     camera_instance = Camera("1", False)
34 else: # 如果输入的地址无效, 返回错误提示
35     return "Invalid source address", 400
36
37 # 返回视频流响应, 调用 video_gen 函数生成视频流数据
38 return Response(video_gen(camera_instance), mimetype='multipart/x-mixed-replace; boundary=frame'
    )

```

在前端, 我们通过 JavaScript 代码处理用户的操作。当用户点击“开始检测”按钮时, 系统会阻止表单的默认提交行为, 获取用户输入的视频源地址, 并构造一个包含时间戳的视频流 URL, 将其设置为图像标签的源地址, 从而实现视频流的加载和显示。如果用户需要切换到另一个摄像头或 RTSP 地址, 只需重新输入新的地址并再次提交请求, 系统将自动停止当前的视频流, 并加载新的视频流进行显示。代码6.4是上述功能的核心片段。

代码 6.4: 表单提交, 加载视频流.js

```

1 document.getElementById('stream-form').addEventListener('submit', function(event) {
2     event.preventDefault();
3     const source = document.getElementById('stream-source').value;
4     const videoStream = document.getElementById('video-stream');
5     videoStream.src = `/video_feed?source=${source}&timestamp=${new Date().getTime()}`;
6 });

```

(3) 报警信息管理功能

报警信息管理功能用于记录和管理检测过程中产生的警报信息。当系统检测到管道缺陷时, 会自动生成带有标注的缺陷照片, 并将其显示在报警信息面板上。用户可以查看警报的详细信息, 包括缺陷类型、位置、严重程度等, 并可以对警报进行确认或标记为已处理。系统支持警报信息的历史记录查询, 方便用户进行后续分析和统计。

报警信息管理功能还具备 PDF 检测报告生成功。它能允许用户将检测结果导出为 PDF 格式的 report。报告中包含检测时间、检测结果等详细信息, 并附带带有标注的缺陷照片, 确保报告的专业性和可读性。PDF 生成技术主要依赖于前端组件协作: 基于 html2canvas 实现页面元素的可视化捕获, 再借助 jsPDF 工具完成 PDF 文件的格式转换

与导出，如图6.6所示。

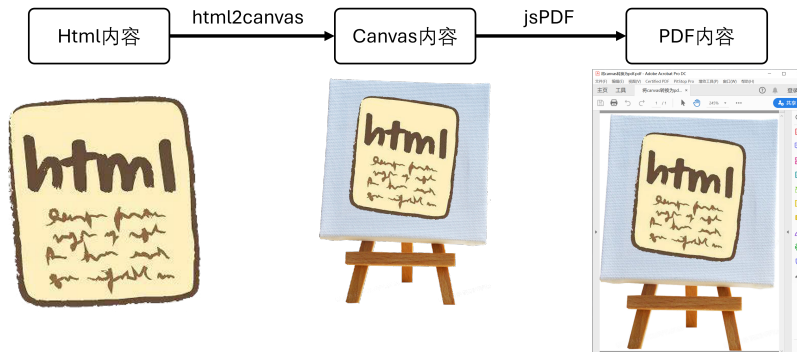


图 6.6 PDF 生成原理

(4) 场站管理功能

在管道检测系统中，场站管理功能为运维效率的提升提供了重要保障。用户可以快速获取场站的名称、位置等重要信息，从而为管道检测工作提供有力支持。为了使场站的位置和分布情况更加直观，本系统在场站管理功能中引入了地理信息标注技术，利用高德地图 API 实现了场站位置的可视化展示，显著增强了场站管理的便捷性与直观性，用户可以在地图上直观地查看场站的位置和分布情况。

代码 6.5: 场站管理.js

```

1  /**
2   * 初始化地图并添加标记
3   * @param {string} dom - 地图容器的ID，默认为'idMap'
4   * @param {Array} center - 地图中心点的经纬度，默认为[120.400456, 30.251112]
5   * @param {string} mapStyle - 地图样式，默认为'normal'
6   * @param {Array} positions - 标记点的经纬度数组
7   * @param {Array} anchor - 标记点的锚点位置数组
8   * @returns {AMap.Map} 初始化后的地图对象
9   */
10 function initMap(dom = 'idMap', center = [120.400456, 30.251112], mapStyle = 'normal', positions =
    [[120.400456, 30.231112], [120.065378, 30.024806], [120.707688, 30.235022], [120.123489,
    30.217567], [119.148932, 29.649969]], anchor = ['top-right', 'top-center', 'top-left', 'middle-
    right', 'center']) {
11 // 创建地图实例
12 let map = new AMap.Map(dom, {zoom: 9, center, resizeEnable: true, mapStyle: `amap://styles/${
    mapStyle}`}),
13     icon = [], marker = [],
14     infoWindow = new AMap.InfoWindow({offset: new AMap.Pixel(0, -30), size: new AMap.Size(300,
    180)});
  
```



```

15
16 // 遍历所有标记点
17 for (let i = 0; i < positions.length; i++) {
18     // 创建标记
19     marker[i] = new AMap.Marker({position: positions[i], icon: icon[i], anchor: anchor[i],
20         offset: new AMap.Pixel(0, 0)});
21     // 创建信息窗口内容
22     let tem = `

在代码6.5中，initMap 函数实现地图初始化功能，通过定义中心点坐标、调整缩放等级及配置可视化模板完成地图创建，同时自动生成站点位置标注信息，用户可以根据需要调整地图的显示效果。positions 参数存储了各个场站的经纬度信息，而 anchor 参数则用于设置每个场站标注的锚点位置。鼠标悬停至特定站点标识时，系统将自动激活信息弹窗，显示该场站的名称和经纬度信息，这为用户提供了更加直观的场站位置参考。



通过这种方式，用户不仅可以快速了解场站的分布位置，还可以方便地获取每个场站的详细信息。这不仅提高了场站管理的效率，也为管道检测工作的规划和实施提供了有力支持。在未来的研究中，还可以进一步优化场站标注的样式和信息展示内容，以满足更多用户的需求。



### 6.2.3 数据字典设计



在系统设计过程中，数据字典用于规范系统中各类基础数据的名称、含义、类型、长度及取值范围等信息，确保各功能模块在数据交互过程中的一致性和准确性。数据元素是数据流、数据存储等数据对象的最小组成单位，其定义的科学性和合理性直接关系到系统的数据处理能力与可扩展性。表6.1对本系统中关键的数据元素进行了详细描述，为后续的数据库设计与系统开发提供基础支撑。



37


```

表 6.1 数据元素的数据字典

编号	数据项名称	含义描述	类型	取值范围	长度
D1-01	image_path	图像路径	string	文件系统路径	≤255
D1-02	defect_type	缺陷类型	string	裂缝/堵塞/腐蚀等	≤20
D1-03	confidence	检测置信度	float	0.0-1.0	4（小数）
D1-04	detect_time	检测时间	datetime	合法时间戳	19
D1-05	camera_id	摄像头编号	string	CAM 编号格式	≤20
D1-06	report_id	警报编号	string	警报编号格式	≤30
D1-07	admin_id	管理员编号	string	用户编码格式	≤20
D1-08	station_id	场站编号	string	场站编码格式	≤20

6.2.4 数据库设计

(1)ER 图设计

在本系统的设计过程中，为明确各实体间的关联性，通过实体-关系图呈现其结构关系，如图6.7所示。

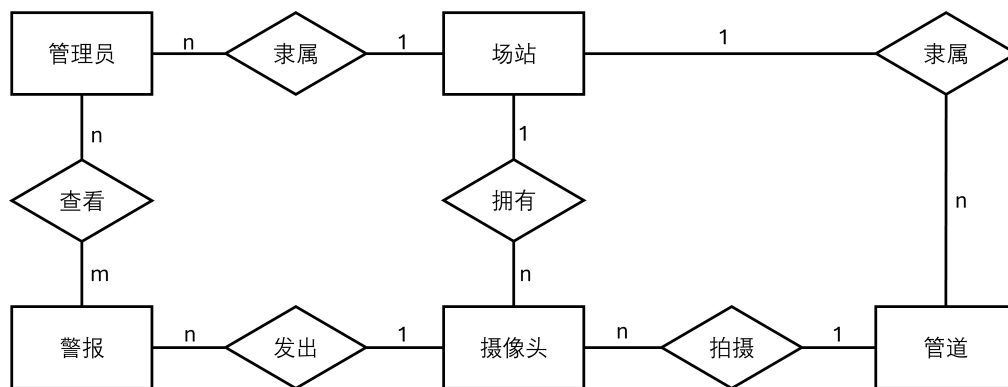


图 6.7 E-R 图

(2) 关系设计

管理员（管理员 id，联系方式，隶属场站 id）

摄像头（摄像头 id，隶属场站，所在管道 id）

管道（管道 id，隶属场站，坐标）

警报（警报 id，摄像头 id，所在位置，时间，缺陷名称）

查看（警报 id，管理员 id，查看时间）

场站（场站 id，联系方式，地址，管道数量，管理员数量，摄像头数量）

6.3 系统实现与展示

6.3.1 开发环境

(1) 硬件环境

本项目的硬件配置对于智能管道缺陷识别系统的运行至关重要，它将直接影响系统的开发效率、调试速度和运行性能。适当的硬件配置不仅能够保证开发和运行效率，还能避免因配置过高而造成的资源浪费。我们的硬件配置能够满足该系统对高效处理和数据管理的需求。具体的硬件配置如表6.2所示。

表 6.2 硬件配置

配置项	配置详情
处理器	Intel® Core™ I7-1165G7 @ 2.80GHz
内存	16GB
硬盘空间	1TB

(2) 软件环境

软件环境搭建以功能实现与系统拓展的双重需求为导向，通过合理配置基础组件实现各模块高效协同。软件环境具体情况如表6.3所示。

表 6.3 软件配置

操作系统	Windows
框架	Bootstrap、jQuery、Flask 等
开发工具	VSCode、IntelliJ IDEA

6.3.2 系统主页面

首页界面是清晰而明确的，如图6.8所示。首页中上部分具有导航栏，可以实现页面跳转。中间部分可以投放一些标语或者产品介绍广告。首页的下半部分设计了一个可以左右滚动的组件，可以放置合作伙伴的标志。整个界面的设计注重实用性和用户体验，确保了信息的即时性和操作的便捷性。

6.3.3 实时监控模块

在本系统中，监控界面的核心功能是提供实时管道摄像头视频流。如图6.9所示，用户可以在页面内观看当前管道摄像头的实时视频画面，以便及时监测管道的状态。当检



图 6.8 首页

测到缺陷时，画面中会出现一个方框标注出缺陷位置和缺陷名称，从而帮助用户快速识别异常情况。监控界面的右侧包含了摄像头选择功能，方便管理者对摄像头进行管控和维护，进一步增强了用户的可操作性。这些功能的集成旨在确保用户能够快速、准确地反应管道异常情况，并采取必要的措施，从而有效提升管道监测的效率和准确性。



图 6.9 实时监控页面

6.3.4 报警信息管理模块

报警信息界面的设计着重于提供全面且高效的监控报警信息展示，以满足用户对快速定位和处理报警事件的需求。为此，采用了列表形式的展示方式，如图6.10所示，使用户能够迅速浏览并定位到特定的报警事件。每条报警信息都详细记录了关键信息，包括触发报警的摄像头位置、报警发生的具体时间、报警类型，以及报警瞬间的视频流截

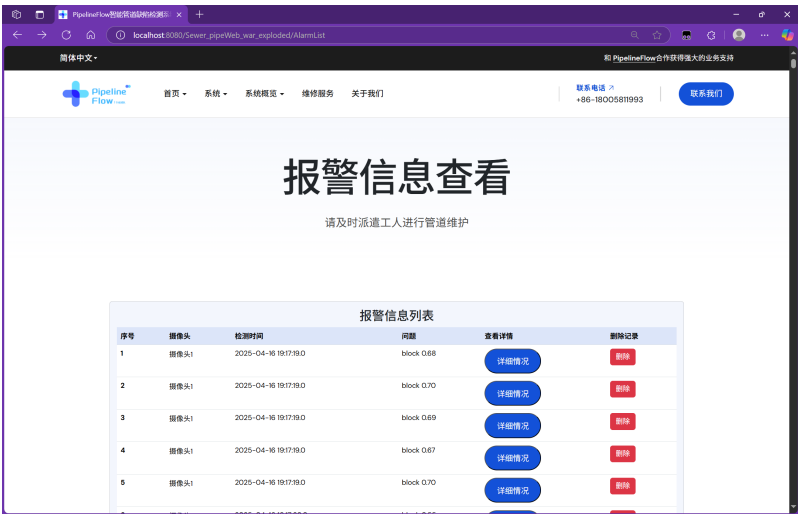


图 6.10 报警信息页面

图。这种设计不仅显著提高了信息检索的效率，而且帮助用户能够迅速理解和响应每个报警事件。

为进一步增强用户体验，报警信息界面还增加了 PDF 检测报告输出功能。用户可以通过该功能将报警信息及相关截图导出为 PDF 格式的报告，便于存档、分享和进一步分析。通过这些综合考虑，报警信息界面旨在为用户提供一个直观、易用且功能全面的工作环境，从而高效地处理和响应各种报警情况，确保监控系统的有效运行和管理。

6.3.5 设备管理模块

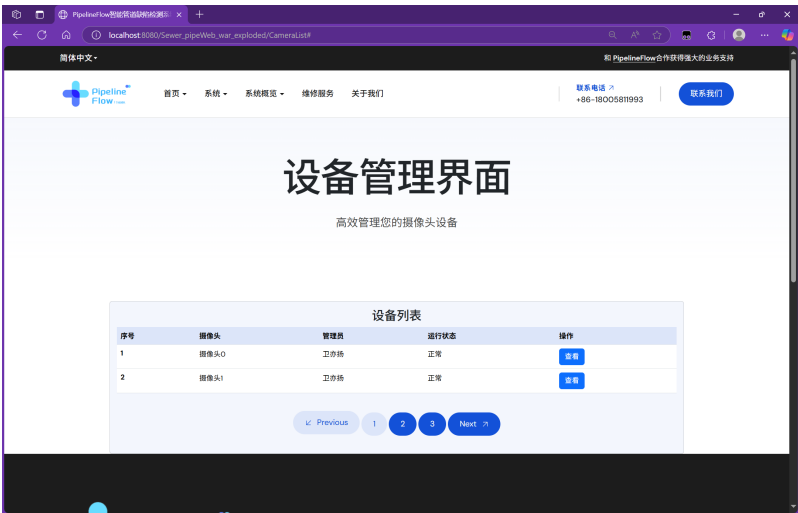


图 6.11 设备管理页面

设备管理界面如图6.11所示。在本系统中，设备管理界面的设计核心聚焦于摄像头管理功能，旨在为用户提供高效、便捷的设备操作体验。其主要功能包括展示摄像头的

详细信息如位置、状态和编号，以及在检测界面进行摄像头的切换。通过该模块，用户可以全面了解摄像头的运行状态，并进行必要的操作，从而确保设备的高效运行。

6.3.6 场站管理模块

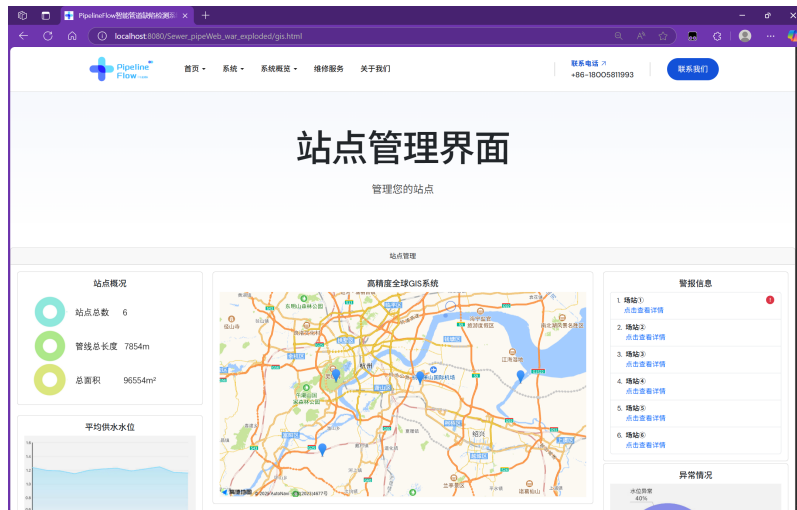


图 6.12 场站管理页面

场站管理页面如图6.12所示。界面采用模块化设计，提升用户体验和操作效率。站点概况模块描述了所有站点的整体情况；水位监控模块展示了一天內站点供排水管道平均水位变化情况；地图系统模块展示了所有站点位置，点击坐标可以查看站点详细信息，用户可以拖动地图界面与系统交互；报警信息模块展示了所有场站摄像头报警情况；异常情况展示了所有场站的管道异常情况。整体设计目标是提供全面、高效的数据管理平台，简化站点和监控设备的管理，提升系统运行效率和数据准确性。

结 论

本项目围绕管道缺陷检测这一现实需求，设计并实现了一个基于深度学习的管道缺陷检测系统。系统通过集成实时监控模块、缺陷识别模型、结果可视化界面和报告生成模块，实现了对管道缺陷的检测与识别，具备较高的检测精度与良好的实时性。本文在模型构建中引入了 AIFI、C3K2_context_guided 和 CBAM 模块，有效提升了模型的特征提取能力与检测效果。在系统实现方面，开发了简洁高效的网页端界面，使用户可以直观查看检测图像和结果，可以生成带有缺陷图片的检测报告，辅助管道管理人员进行维护决策。通过本项目的开发与测试，验证了深度学习目标检测技术在实际工程问题中的可行性和应用价值，为管线的智能化巡检提供了新思路与解决方案。

面向未来，随着智慧城市理念的不断深化，城市基础设施管理正亟需更多智能化、自动化手段支持。为了更好地服务于行业发展，本项目将进一步扩展功能并优化系统性能。具体来说，我计划部署多个检测模型于服务器端，提升并发检测能力和模型切换灵活性；采用 Ceph 等分布式存储系统解决海量数据存储与调用问题；同时引入更先进的摄像头与传感器设备，实现更全面的数据采集与处理。此外，未来还可将水质监测、管道修复建议等功能集成至系统中，逐步构建涵盖“检测-分析-决策-管理”的全流程智能平台。相信随着技术的不断发展与完善，基于人工智能的管道缺陷检测系统将为城市供排水安全运维带来更广阔的前景与更坚实的支撑。

参考文献

- [1] Vilkys T, Rudzinskas V, Prentkovskis O, et al. Evaluation of failure pressure for gas pipelines with combined defects[J]. *Metals*, 2018, 8(5): 346.
- [2] Gao Q, Yu X, Jiang H, et al. Local corrosion behavior of pipeline steel under deposition layer in produced water of Alkali/Surfactants/Polymers[J]. *Colloids and Surfaces A: Physicochemical and Engineering Aspects*, 2023, 679: 132609.
- [3] Wang L, Mao Z, Xuan H, et al. Status diagnosis and feature tracing of the natural gas pipeline weld based on improved random forest model[J]. *International Journal of Pressure Vessels and Piping*, 2022, 200: 104821.
- [4] Xu L, Dong S, Wei H, et al. Intelligent identification of girth welds defects in pipelines using neural networks with attention modules[J]. *Engineering Applications of Artificial Intelligence*, 2024, 127: 107295.
- [5] Bhagat A, Basireddy S R. Vision Based Automated Pipeline Inspection[C]//2024 18th International Conference on Control, Automation, Robotics and Vision (ICARCV). 2024: 789-794.
- [6] Wang L, Guo W, Guo J, et al. An integrated deep learning model for intelligent recognition of long-distance natural gas pipeline features[J]. *Reliability Engineering & System Safety*, 2025, 255: 110664.
- [7] Zhao Q, Wang G. Multi-architecture optimization of pipeline inner wall defect detection algorithm based on YOLOv8[J]. *Measurement*, 2025, 242: 116305.
- [8] Xiao J, Xu L, Li C, et al. Lightweight visible damage detection algorithm for embedded systems applied to pipeline automation equipment[J]. *Journal of Pipeline Science and Engineering*, 2025: 100254.
- [9] Zhao Y, Lv W, Xu S, et al. Detrs beat yolos on real-time object detection[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2024: 16965-16974.
- [10] Wu T, Tang S, Zhang R, et al. Cgnet: A light-weight context guided network for semantic segmentation[J]. *IEEE Transactions on Image Processing*, 2020, 30: 1169-1179.

- [11] Woo S, Park J, Lee J Y, et al. Cbam: Convolutional block attention module[C]// Proceedings of the European conference on computer vision (ECCV). 2018: 3-19.
- [12] 辛佳兴, 陈金忠, 李晓龙, 等. 油气管道内检测技术研究前沿进展[J]. 石油机械, 2022, 50(5): 119-126.
- [13] 李飞. 基于深度学习的管道缺陷检测方法研究[D]. 中国矿业大学, 2023.
- [14] Rayhana R, Yun H, Liu Z, et al. Automated defect-detection system for water pipelines based on CCTV inspection videos of autonomous robotic platforms[J]. IEEE/ASME Transactions on Mechatronics, 2023, 29(3): 2021-2031.
- [15] 胡群芳, 郑泽昊, 刘海, 等. Application of 3D ground penetrating radar to leakage detection of urban underground pipes[J]. 同济大学学报 (自然科学版)(英文版), 2020, 48(7): 972-981.
- [16] 郑州九泰科技有限公司. 排水管道缺陷检测发展现状及展望[EB/OL]. 郑州九泰科技有限公司. 2024 [2024-10-12]. https://www.jtkjnkj.com/news_detail/71.html.
- [17] Hussain M. YOLO-v1 to YOLO-v8, the rise of YOLO and its complementary nature toward digital manufacturing and industrial defect detection[J]. Machines, 2023, 11(7): 677.
- [18] Jia P, Guo T, Guo F, et al. Detection model of drainage pipe defect based on improved YOLOv5[C]//2023 8th International Conference on Intelligent Computing and Signal Processing (ICSP). 2023: 1950-1955.
- [19] Jegham N, Koh C Y, Abdelatti M, et al. Evaluating the evolution of yolo (you only look once) models: A comprehensive benchmark study of yolo11 and its predecessors[J]. arXiv preprint arXiv:2411.00201, 2024.
- [20] Khanam R, Hussain M. YOLOv11: An overview of the key architectural enhancements [J]. arXiv preprint arXiv:2410.17725, 2024.
- [21] Wang T, Li Y, Zhai Y, et al. A sewer pipeline defect detection method based on improved YOLOv5[J]. Processes, 2023, 11(8): 2508.
- [22] Yan W, Liu W, Bi H, et al. Yolo-pd: Abnormal signal detection in gas pipelines based on improved yolo v7[J]. IEEE Sensors Journal, 2023, 23(17): 19737-19746.
- [23] Yang D, Ma C, Yu G, et al. Automatic defect detection of pipelines based on improved OFG-YOLO algorithm[J]. Measurement, 2025, 242: 115847.
- [24] Peng D, Pan J, Wang D, et al. Research on oil leakage detection in power plant oil

depot pipeline based on improved YOLO v5[C]//2022 7th International Conference on Power and Renewable Energy (ICPRE). 2022: 683-688.

[25] Tang J, Liu S, Zhao D, et al. An Improved Detection Algorithm of PCB Surface Defects Based on YOLOv5., 2023, 15, 5963[J]. DOI: <https://doi.org/10.3390/su15075963>, 2023.

[26] Wen Z, Liu J, Shen X, et al. A Lightweight Pipeline Defect Detection Method via Structural Reparameterization Technique and Ghost Convolution[C]//2023 IEEE 12th Data Driven Control and Learning Systems Conference (DDCLS). 2023: 723-727.

[27] Zhu C, Ye H. A modular method for GPR hyperbolic feature detection and quantitative parameter inversion of underground pipelines[J]. Remote Sensing, 2023, 15(8): 2114.

[28] Seghier M E A B, Mohamed O A, Ouaer H. Machine learning-based Shapley additive explanations approach for corroded pipeline failure mode identification[C]//Structures: vol. 65. 2024: 106653.

[29] Zhang W, Zhang Z, Yao B, et al. High-density interwoven pipeline leak detection with high-sensitivity and high-resolution quartz pressure transducer[J]. Mechanical Systems and Signal Processing, 2025, 225: 112255.

[30] 苏德尔, 李浩宇, 高伟达, 等. 用于管道检测机器人的微型化成像系统 (特邀)[J]. Laser & Optoelectronics Progress, 2024, 61(2): 0211013-0211013.

[31] De S Thiago Filho A M, Amaral I F S, de S Thiago Neto N C P, et al. Robotic Pipe Inspection: Low-Cost Device and Navigation System[J]. Journal of Control, Automation and Electrical Systems, 2025, 36(1): 101-116.

[32] Haurum J B, Moeslund T B. Sewer-ML | vap.aau.dk[EB/OL]. Aalborg University. n.d. [2023-10-01]. <https://vap.aau.dk/sewer-ml/>.

[33] Coisini. Pipe Object Detection Dataset and Pre-Trained Model by Coisini[EB/OL]. Coisini. 2023 [2023-10-01]. <https://universe.roboflow.com/coisini-bangq/pipe-prw2b>.

[34] Ultralytics. Ultralytics YOLO11 - Ultralytics YOLO 文档[R/OL]. Ultralytics. 2023 [2023-12-31]. <https://docs.ultralytics.com/zh/models/yolo11/>.

[35] Redmon J, Farhadi A. Yolov3: An incremental improvement[J]. arXiv preprint arXiv:1804.02767, 2018.

[36] Jocher G, Chaurasia A, Stoken A, et al. ultralytics/yolov5: v7. 0-yolov5 sota realtime instance segmentation[J]. Zenodo, 2022.

[37] Ren S, He K, Girshick R, et al. Faster r-cnn: Towards real-time object detection with region proposal networks[J]. Advances in neural information processing systems, 2015, 28.

[38] Duan K, Bai S, Xie L, et al. Centernet: Keypoint triplets for object detection[C]// Proceedings of the IEEE/CVF international conference on computer vision. 2019: 6569-6578.

[39] 中研普华, 中研网. 2025 年管道检测修复行业发展现状、市场规模与产业链分析[EB/OL]. 中研普华. 2025 [2025-04-11]. <https://www.chinairn.com/scfx/20250411/161530844.shtml>.

[40] 人人文库. 管道检测行业现状分析报告[R/OL]. 人人文库. 2024 [2024-03-24]. <https://www.renrendoc.com/paper/320498181.html>.

致 谢

四年的大学时光倏忽而过，转眼我站在了本科生涯的终点。这篇毕业论文的完成，凝聚了我无数个日日夜夜的思考与努力，也离不开诸多良师益友的倾力支持和无私帮助。在此，我怀着感恩之心，向所有给予我支持、鼓励和指导的人们致以最诚挚的谢意。

首先，我要特别感谢我的导师朱凌教授。感谢您在我学业生涯中给予的悉心指导和无私帮助。从课题选定到论文框架搭建，从算法调试到系统打磨，您的耐心指引与专业素养如灯塔般照亮我的前行。每次遇到瓶颈时，都是您的鼓励让我重拾信心，您的细致讲解让我看到了更广阔的视野。我始终记得第一次向您展示我的研究思路时，您认真听我讲完并提出建议，那一刻我感受到了温暖和支持，也坚定了要做好这项工作的信念。感谢您不仅教我如何做科研，更教会了我如何做一个沉稳、踏实、有责任感的人。

其次，感谢浙江财经大学及信息技术与人工智能学院，感谢母校为我提供的优质学习环境和丰厚资源，让我能够探索自己热爱的领域，并逐渐明确前行的方向。四年的大学生活让我遇见了许多优秀的老师和同学，尤其是赵梦瑶老师，作为我的支部书记，您给予我无私的帮助和关怀。在我遇到困惑时，您总是耐心解答、温暖安慰，让我在迷茫中找到方向，在低谷中重新振作，坚定了我前行的信念。

此外，我要感谢我的同学和朋友们。在这段科研旅程中，是你们的陪伴让我充满动力与勇气，支持让我深刻体会到团队与友谊的珍贵。感谢李想学长的耐心指导与经验分享，让我少走了许多弯路；感谢我的室友吴俊宁和徐浩铭、班长江恬恬，四年来我们彼此陪伴，度过了许多难忘的时光；感谢严静怡同学，在竞赛中并肩作战的日日夜夜，你的坚持和努力给了我莫大的力量。

最后，我要将最深的感激献给我的家人。感谢你们一直以来的支持与陪伴，是你们给了我最坚强的后盾和最温暖的依靠，让我在迷茫和疲惫时能够重新振作，勇敢前行。

感谢所有出现在我生命中的人。于万人中万幸得以相逢，你们见证了我从懵懂青涩逐步走向成熟坚定，也给予了我无数勇气与力量，让我在追梦路上从未孤单。毕业不是终点，而是新的起点。行文至此，虽有很多不舍，但也该为我的本科学习生活画上句号。论文的完成标志着一个阶段的落幕，但学术的道路仍需孜孜以求，人生的征程也才刚刚启程。我将带着这份沉甸甸的感激，继续奔赴热爱、追逐光明，不负青春，不负众望。

谨以此文，致谢所有在我大学旅途中播下光亮的人。